

**Web-Based Intelligent Driving School Management and Enrollment
System with Automated Instructor Matching and Performance
Analytics for BLRT**

**A Capstone Project Presented to the Faculty of the
Information Technology Department
Binalatongan Community College
San Carlos City, Pangasinan**

**In Partial Fulfillment of the Requirements
for the Degree of Bachelor of Science
in Information Technology**

Submitted by:

**Cabuang, Joshua Q.
Jacla, Ryan M.
Casay, Jim Spencer Lee C.
Palisoc, Romark D.**

May 2026



APPROVAL SHEET

This capstone project entitled **Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT** prepared and submitted by **Joshua Q. Cabuang, Ryan M. Jacla, Jim Spencer Lee C. Casay, Romark D. Palisoc** in partial fulfillment of the requirements for the degree **BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY**, has been examined and is recommended for acceptance and approval.

MR. FRANCIS L. CRISOSTOMO, BSIT

Adviser

PANEL OF EXAMINERS

BANNER B. FERRER, MIT

Chairperson

CHARLES NIXON C. CAYANDING, BSIT

Member

JANNELE M. DE VERA, MIT

Member

ACCEPTED and **APPROVED** in partial fulfillment of the requirements for the degree **BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY** on **May 17, 2022** with a grade of _____.

BRIXON J. DE VERA, MIT

Dean, Information Technology Department

DR. MACRINA B. CAJALA

College President



Abstract

Joshua Q. Cabuang, Ryan M. Jacla, Jim Spencer Lee C. Casay, Romark D. Palisoc, "Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT," Bachelor of Science in Information Technology, College of Information Technology, Binalatongan Community College, San Carlos City, Pangasinan, Philippines, May 2025.

Adviser: **FRANCIS L. CRISOSTOMO, BSIT**

This study addressed the critical operational bottlenecks of BLRT Driving School Inc. in San Carlos City, Pangasinan, an institution that historically relied on manual record-keeping, handwritten logbooks, and physical scheduling boards to coordinate its licensing courses. This traditional paradigm frequently exposed the establishment to severe administrative vulnerabilities, including instructor double-bookings, human error in tracking mandatory training hours, data redundancies, and communication delays that overwhelmed personnel. To mitigate these systemic inefficiencies, the researchers designed and developed the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics. Adopting the Rapid Application Development (RAD) methodology—which systematically progressed through the iterative phases of requirements planning, user design, rapid construction, and cutover—the system was engineered using a robust full-stack architecture consisting of Laravel, Livewire, Tailwind CSS, and Alpine.js (the TALL stack), backed by a MySQL database management system.

The completed platform integrates a secure, digitized enrollment pipeline with automated document verification, a responsive chatbot assistance module for recurring inquiries, an intelligent scheduling algorithm that automatically pairs



students with accredited instructors based on real-time availability and specialization, and an advanced performance analytics dashboard for data-driven managerial oversight. Rigorous empirical evaluations demonstrated excellent technical efficacy: functionality testing achieved an 86% success rate under multi-role access simulation, while performance and security testing both achieved a flawless 100% success rate, validating structural immunity against severe web vulnerabilities such as SQL injection, Cross-Site Scripting (XSS), Insecure Direct Object Reference (IDOR), and malicious file uploads. Furthermore, the platform attained exceptional marks in usability testing ($M = 4.73$) and user acceptance testing ($M = 4.50$), with both metrics yielding a descriptive interpretation of "Strongly Agree" from stakeholders and end-users. Ultimately, the study concludes that the developed system successfully replaces error-prone manual labor with a secure, highly scalable computing solution that optimizes resource allocation and safeguards data integrity, serving as a definitive benchmark for digital transformation within specialized vocational training environments.



Acknowledgement

The completion of this capstone project would not have been possible without the guidance, support, and contributions of numerous individuals. The researchers extend their deepest gratitude to those who shared their time, expertise, and encouragement throughout this rigorous academic journey.

First and utmost, the researchers express their profound gratitude to the Almighty God, the source of all wisdom, strength, and perseverance. His divine providence sustained the team through moments of technical complexity, granting them the clarity of mind necessary to bring this study to fruition.

The researchers extend their sincere appreciation to the administration of Binalatongan Community College, particularly to the Dean and Department Chairperson of the Institute of Computing Studies, for establishing an academic environment that fosters innovation and technical excellence.

Deepest gratitude is expressed to Francis L. Crisostomo, their capstone adviser, whose invaluable mentorship, constructive criticisms, and meticulous guidance shaped the architecture and technical execution of this platform.

The team also gives credit to the esteemed members of the Panel of Examiners. Their insightful evaluations, stringent reviews, and professional recommendations during the defense phases greatly enhanced the security, performance, and overall utility of the developed system.

Special thanks are extended to the management and staff of BLRT Driving School Inc. in San Carlos City, Pangasinan. Their cooperation and active participation during data gathering and system evaluation were vital to transforming this technological solution into an operationally viable asset.



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



The researchers express their profound love and gratitude to their respective parents, guardians, families, and loved ones. Their unconditional love, moral guidance, and financial support served as the foundational pillars that sustained the team throughout this endeavor.

Finally, this accomplishment stands as a testament to the shared dedication, camaraderie, and synergy of **Team B1**. The mutual trust and collaborative spirit maintained among the researchers ensured the triumphant execution of this project.

The Researchers:

Joshua Q. Cabuang

Jim Spencer Lee C. Casay

Ryan M. Jacla

Romark D. Palisoc



Table of Contents

	Page
Title Page	i
Approval Form	ii
Abstract	iii
Acknowledgment	v
Table of Contents	vii
List of Tables	ix
List of Figures	x
 Chapter	
1 INTRODUCTION	1
Background of the Study	1
Purpose and Description	5
Objectives of the Study	7
Conceptual Framework (IPO)	8
Scope and Limitations	11
Definition of Terms	14
Review of Related Literature.....	16
 2 METHODOLOGY AND DESIGN	21
Software Development Methodology	22
Sources of Data	25
Data Gathering Techniques.....	27
System Design	29
System Architecture	29
Dataflow Diagram	38
Entity-Relationship Diagram	34
Implementation Diagram	41
 3 RESULTS AND DISCUSSION	45
System Process Flowchart.....	47
System Features and User Interfaces	51
System Testing and Evaluation	72
Implementation Results	76
Analysis of Results	81
Discussion of Findings	78
 4 RECOMMENDATIONS.....	#



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan
Email Address: bcc.sccp@yahoo.com



Appendices	95
A Endorsement Letter	95
B System Source Codes	#
C Database Schema (Actual Database Structure)	#
D Survey/Evaluation Forms Used During Testing	#
E Collected Sample Documents for Document Analysis/Data Gathering.....	#
Curriculum Vitae	#



List of Tables

Table	Title	Page
	System development methodology	23
3.24	Functional Testing	82
3.25	Performance Testing	84
3.26	Security Testing	86
3.27	Usability Testing	89
3.28	User Acceptance Testing	92



List of Figures

Figure	Title	Page
1.1	Input Process Output	8
2.1	System Architecture	29
2.2	Existing Process Flowchart	33
2.2	Entity Relationship Diagram	34
2.3	Date Flow Diagram	38
2.24	Project Timeline	42
3.1	Enrollment and Verification Process	47
3.2	Registration and Verification	48
3.3	Chatbot Process	49
3.4	User Verification and Approval	52
3.5	Document Review and Management	53
3.6	Enrollment Oversight	54
3.7	Role-Based Access Control Management	55
3.8	Course Management	56
3.9	Vehicle Management	57
3.10	Performance Analytics Dashboard	58
3.11	Enrollment Validation	59
3.12	Schedule management	60
3.13	Instructor and Student Coordination	61
3.14	Schedule Viewing and Management	62
3.15	Student Progress Monitoring	63
3.16	Attendance Recording	64
3.17	Performance Evaluation and Grading	65
3.18	Online Registration and Enrollment	66
3.19	Document submission	67
3.20	Course Scheduling Access	68
3.21	Intelligent Instructor Marching	69
3.22	Progress Tracking	70
3.23	Chatbot Assistance	71
C.1	Users	133
C.2	Student_Profiles	133
C.3	Instructor_Profiles	134
C.4	Instructor_Metrics	134
C.5	Assessments	135
C.6	Booking_Sessions	135



Figure	Title	Page
1.1	Input Process Output	8
C.7	Courses	136
C.8	Documents	136
C.9	Enrollment_Forms	137
C.10	Enrollments	137
C.11	Instructor_Performances	138
C.12	LTO_Clinics	138
C.13	Model_Has_Permissions	139
C.14	Model_Has_Roles	139
C.15	Permissions	140
C.16	Role_Has_Permissions	140
C.17	Roles	141
C.18	System_Matrices	141
C.19	Vehicles	142
C.1	BLRT Driving School Manual Registration Form	146
C.2	BLRT Driving School Practical DrIving Course (PDC) Training Log	147
C.3	BLRT Driving School Practical Driving Assessment Report	148



Chapter I

INTRODUCTION

Background of the Study

In the modern digital landscape, computing solutions such as web-based applications and integrated management systems have become important tools for enhancing operational efficiency and addressing complex administrative challenges. The transition from manual processes to automated platforms allows organizations to achieve higher data accuracy, streamlined workflows, and improved user engagement. In specialized educational sectors, such as technical vocational institutions and driving schools, the integration of intelligent scheduling and data analytics is crucial for managing high volumes of student records and coordinating specialized human resources like accredited instructors.

According to Ajiga (2024) in the study called "The Role of Software Automation in Improving Industrial Operations and Efficiency " software automation is a tool that can change old manual processes into efficient workflows. The study says that by using automated systems of people for repetitive and time-consuming tasks organizations can speed up their operations and make fewer mistakes that lead to uneven results. Software automation is good because it can analyze data in time and manage resources better which helps make decisions based on facts. The research also notes that automation is crucial for staying with competitors because it makes operations reliable and increases productivity. This study supports the creation of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT as it shows that manual scheduling and logbooks should be replaced with a system that ensures accurate data and efficient service. The Web-Based Intelligent Driving School.



Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT needs software automation to work well. Software automation helps to make operations faster and more reliable. It also helps to make decisions with accurate data. The study by Ajiga (2024) highlights the importance of software automation in improving operations. The Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics, for BLRT will benefit from software automation.

In Riño and Daing's (2024.) entitled "Challenges in Handling Student Records and Characteristics of Student Information Management System in Public Secondary School in Marilao South District IV Bulacan," underscores the persistent difficulties educational institutions face in the Philippines regarding manual data management. Their research highlights that teachers experience significant challenges in data entry and the secure saving of student information, often leading to administrative bottlenecks. The study emphasizes that the implementation of a centralized registry—similar to the Department of Education's Learner Information System (LIS)—is essential for accurate tracking and informed decision-making. Moreover, the authors stress that any modern management system must prioritize data security in accordance with the Data Privacy Act of 2012. These findings directly mirror the current situation at BLRT Driving School, where the reliance on manual record-keeping hampers operational speed and highlights the urgent need for a web-based solution that ensures data integrity and streamlined student tracking.



The focus of this study is BLRT Driving School Inc. a leading driving educational institution in San Carlos City, Pangasinan. BLRT is the only driving school in the city and the school provides essential Driver's License training courses, including the mandatory 15-hour Theoretical Driving Course (TDC) and Practical Driving Courses (PDC) for motorcycles, tricycles, and sedan motor vehicles (both manual and automatic). BLRT is recognized by the Department of Transportation (DOTr) and Land Transportation Office (LTO), the institution has highly trained and accredited professionals that produce discipline and responsible drivers.

Regardless of its success and local standing, BLRT Driving School currently relies heavily on traditional, manual methods for managing its core operations. The institution uses physical boards and manual logbooks for scheduling driving lessons and tracking student hours. This manual approach led BLRT Driving School to operational difficulties, especially the high risk of double-booking instructors or vehicles. Furthermore, tracking the completion of mandatory training hours for numerous students across different vehicle categories is time-consuming and prone to human error. Also, the student can not view their schedules or track their progress, this will lead to more inquiries that overwhelm the staff.

The manual Enrollment creates issues in management of data. The current processes of BLRT Driving School are done through physical forms leading to difficulty generating reports. The absence of an automated instructor matching system means that scheduling is often conducted through manual writing on a physical board and paper-based logbooks rather than an optimized assessment of instructor specialization, availability and student needs. These identified gaps highlight an urgent need for a modern solution that can automate BLRT Driving School's core operations.



To bridge these gaps, the researchers propose the development of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT. This computing solution is designed to replace the existing manual workflows with an integrated digital platform featuring automated instructor matching and a performance analytics engine. By making the enrollment process online and utilizing a specialized algorithm for scheduling, the system will eliminate the risks of double-booking and ensure an equitable distribution of teaching loads. The dashboard for analytics will empower the management of BLRT to make data-driven decisions and enhancing the efficiency of their operations and the quality of service provided to the community of San Carlos City.



Purpose and Description

The purpose of this project is to develop a Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT. This system aims to create a comprehensive web-based platform that automates and simplifies BLRT Driving School's primary operational procedures such as enrollment, scheduling instructor assignment and performance tracking. By minimizing manual errors, reducing administrative workload, and enhancing communication between students, teachers and administrative staff the system aims to increase organizational efficiency.

The platform's intelligent instructor matching feature seeks to improve training quality and the overall learner experience by matching students with the best instructors based on specialization, availability, and training requirements. The purpose of integrating a chatbot is to handle recurring questions, encourage quicker response times, and offer reliable requirements information. Ultimately, the system aims to support BLRT Driving School in modernizing its operations through digital transformation, enabling more accurate record-keeping, data-driven decision making, and improved service delivery.

Once the proposed Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT is implemented at BLRT Driving School, it will hold particular significance for the following beneficiaries

Driving School Administration - The system will serve as a centralized platform for managing enrollee records, automating scheduling conflicts. This directly improves operational efficiency and reduces data redundancy.



Student Applicants - They will benefit from a streamlined enrollment process, real-time access to available schedules, and transparent tracking of their course progress and payment status.

Developers/Future Developers - They will benefit from a well-documented and modular system architecture that allows easier feature updates, debugging, and scalability. The platform supports efficient collaboration, faster deployment of new functionalities, and long-term system improvement through continuous development and optimization.

Driving Instructors - The system provides them with accurate, real-time daily schedules and a digital platform to input student grades and remarks, replacing manual logbooks.

Parents and Guardians - The system provides assurance of legitimate scheduling and payment transparency, allowing them to verify that the student is attending the authorized 15-hour seminars and practical sessions.

The rationale behind the project is rooted in addressing the shortcomings of the current manual system inefficient tracking, delayed communication. By replacing the automated approach with a centralized web-based platform, the project assumes that the new system will significantly improve process transparency and stakeholders' satisfaction.



Objectives of the Study

The main objective of the study is to design and develop a Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT Driving School Inc. in San Carlos City, Pangasinan. The purpose of this system is to make the core operations of BLRT Driving School Inc. form manual, paper and board-based to a digital platform that helps the school in decision-making and enhances their workflow.

Furthermore, the developers aim to achieve the following specific objectives:

1. To analyze the existing processes of student enrollment, manual instructor scheduling, and student performance tracking at BLRT Driving School Inc. to identify inefficiencies, challenges, and opportunities for improvement.
2. To identify the key features of the system for the following user roles:
 - **Management:** To access the performance analytics dashboard for monitoring business growth, instructor efficiency, and overall school operations.
 - **Staff/Administrators:** To manage student registrations, check payments, and oversee the automated scheduling engine to prevent double-booking.
 - **Instructors:** To view their assigned schedules in real-time, manage student attendance, and input grades for Theoretical (TDC) and Practical (PDC) courses.
 - **Students:** To perform online enrollment, upload necessary documents, view their schedules, and track their progress.



3. To test and evaluate the system's functionality, performance, security, usability, and acceptability to ensure it meets user requirements and the standards of the driving school industry.
4. To prepare an implementation plan for the deployment of the system, including data migration from manual logs and the training of BLRT staff and instructors.



Conceptual Framework

The study will be utilizing Input-Process-Output(IPO) that organizes the specific elements required to transform a manual business problem into a technology-based solution. This model serves as the research roadmap, ensuring that all technical requirements and developmental stages are strategically aligned to produce a reliable and efficient system for BLRT Driving School.

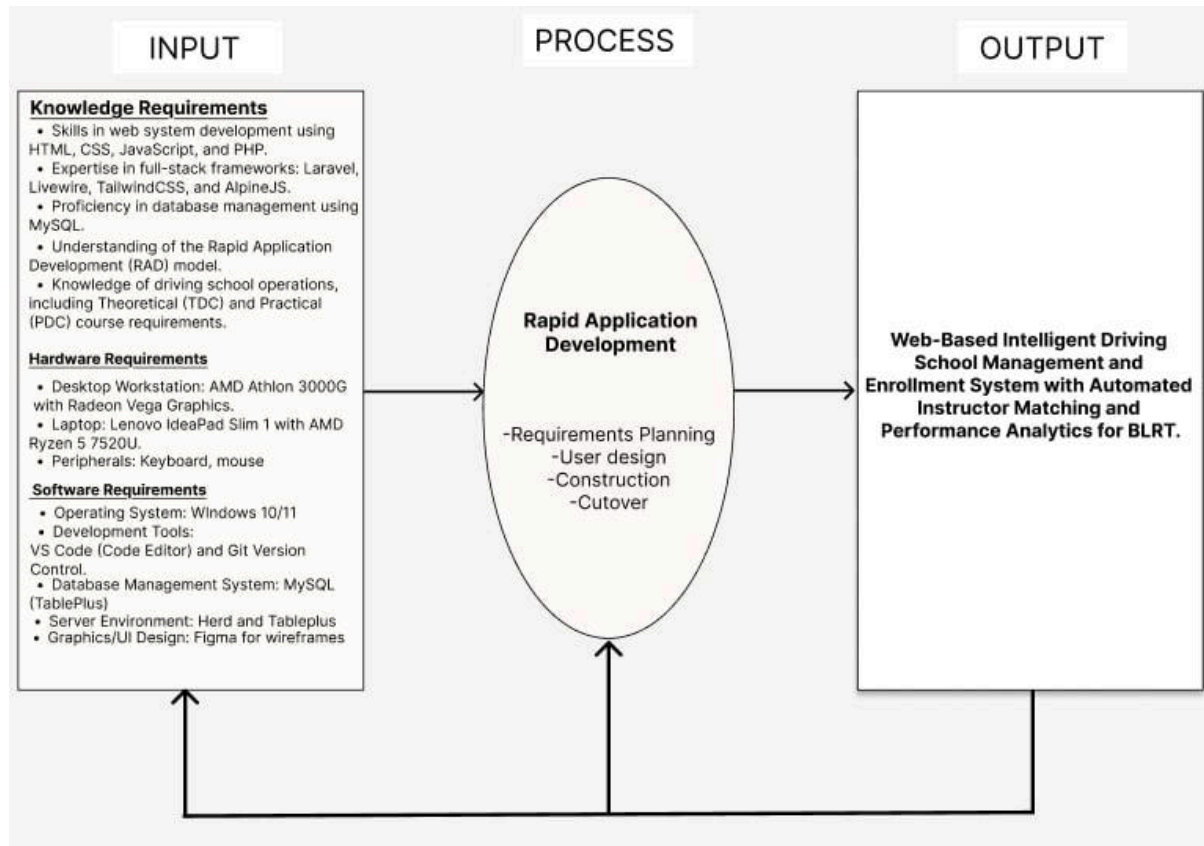


Figure 1.1 IPO



The Input phase consists of the important resources needed to start development. This includes Knowledge Requirements, specifically technical proficiency in full-stack web development using PHP (Laravel), Livewire, TailwindCSS, and AlpineJS, as well as domain knowledge regarding Theoretical (TDC) and Practical (PDC) course structures. Hardware Requirements involve the use of a desktop workstation (AMD Athlon 3000G) and a Lenovo IdeaPad Slim 1 laptop for coding and testing. Software Requirements consist of Visual Studio Code, Git for version control, Herd and TablePlus for local hosting, and Hostinger Philippines for the final deployment and domain management.

The Process phase outlines the execution of the Rapid Application Development (RAD) methodology. This iterative approach begins with Requirements Planning, where the developers identify the specific bottlenecks in BLRT's manual board-based scheduling. It moves into User Design, where wireframes and prototypes of the enrollment and analytics dashboards are created based on stakeholder feedback. This is followed by Rapid Construction, the core coding phase where the important features and database architecture are built. Last is the Cutover stage, which involves final system testing, migrating records from manual paper and board-based to the MySQL database, and training the BLRT staff.

In the Output phase, the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT is expected to be functional. This output shows the successful integration of all inputs into the RAD process, resulting in an automated platform that prevents human errors, simplifies student enrollment,



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



and provides management with actionable data via a performance analytics engine.



Scope and Limitations

Scope

This section will specify the scope and basic functionalities of the proposed Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT. This system is utilized to automatically manage the core operational processes of the driving school such as enrollment, schedules, checking the availability of instructors, and monitoring of student performance.

The system will replace the current manual paper-based assessments and school scheduling boards in the management of student records. The system will be web-based which may be accessed by web browsers on mobile phones and desktop computers with an active internet connection or mobile data connection.

The proposed system is a web-based platform developed using Visual Studio Code and the Laravel framework, utilizing Livewire, TailwindCSS, and Alpine.js to ensure a responsive and efficient development lifecycle. PHP serves as the primary back-end language, with MySQL utilized for data management and Git for version control and collaborative tracking. To ensure public accessibility, the application will be hosted via Hostinger Philippines using a registered domain name. The system identifies five distinct user roles such as Admins, Staff, Driving Instructors, and Students. Each with specific access levels for operational management and reporting. Finally, a dedicated Student Portal facilitates the digitized enrollment process, allowing learners to sign up, submit documentary requirements, and register for Theoretical (TDC) and Practical (PDC) Driving Courses.



Students can check their progress, and using an integrated Chatbot Assistance Module, make quick inquiries about rates for courses, required requirements and enrollment procedures. Administrators and staff will have all the control over user administration, including verification of registrations, viewing of uploaded documents, monitoring of master schedules and vehicles.

A fundamental feature of the system is an intelligent instructor matching module, that can automatically assign an instructor to a student based on the instructor's specialization and availability. Instructors can log in to see their assigned students and their daily teaching schedule, record attendance, score practical driving skills and validate final grades and remarks directly into the system. Furthermore, the school managers will have performance analytics and a reporting dashboard where they can visualize enrollment trends, instructor workloads and student performance. School managers will also benefit from performance analytics and a reporting dashboard that visualizes enrollment trends, instructor workloads, and student performance.

Limitations

This section outlines the boundaries of the system's functionality to establish realistic expectations and clearly define areas excluded from the initial development scope.

While the suggested Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT encompasses extensive automation of the operational workflow at BLRT Driving School, it has certain limitations to keep the project grounded. Primarily, the system is a web-based application. As such, it can only be accessed through a web browser on mobile devices, laptops or desktop



computers when an internet connection or mobile data is available. It does not provide a mobile application to function without internet connection.

Regarding integrations with external systems and online financial transactions, the system operates independently of the Philippine government's Land Transportation Office (LTO) network. While the system provides visibility to track a student's payment status, it does not have an integrated online payment gateway for real-time digital bank transfer or e-wallet transactions. Additionally, the system's advanced features are restricted to specific parameters. The intelligent instructor matching algorithm will pair students with instructors based on the system's record of an instructor's specialization and availability. The integrated Chatbot Assistance Module can only respond to frequently asked questions about course rates, enrollment requirements and how to enroll. The system does not provide complex conversational AI features, live-chat with human staff, or personalized driving instruction.



Definition of Terms

The researchers have defined some terms that they want to explain. They do this so that the people reading this document know what these terms mean. The researchers want to make sure that everyone is on the page.

Theoretical Driving Course (TDC) - is a 15-hour seminar that people need to take if they want to get a Student Permit.

The Practical Driving Course (PDC) - is hands-on training for people who want to get a Non- Professional drivers license.

The Intelligent Instructor Matching System - is a feature that matches students with instructors who're a good fit for them. It looks at what the instructors specialize in when they're available and what the students need to learn.

The Chatbot Assistance Module - is a tool that answers questions that people ask a lot. It gives answers to questions about how much things cost, how to sign up and what people need to do.

The Performance Analytics Module - is something that the school management uses to make reports and charts about things like how many people're signing up, how busy the instructors are and how well the students are doing. This helps the school make decisions.

The Rapid Application Development (RAD) Method - is how the researchers made the software. The researchers worked on it a little at a time, asked the users what they thought and made changes based on what people said.



The Admin or Super Administrator - is the person in charge of the system. They check to make sure new students and instructors are registered correctly and they look at the documents that people upload. They manage who is signed up for what. They can also give people roles in the system.

The Student Monitoring and Performance Tracking System - keeps track of things like who's attending classes, how far along people are, in their training and when people finish their sessions. This works for both the Theoretical Driving Course (TDC) and the Practical Driving Course (PDC).



Review of Related Literature

The first step to modernizing schools and training centers is to switch from using paper to systems. Some researchers like Kumaran and his team found out that using computers to manage student files and cars can really help reduce mistakes and lost data. They think it is very important to have a place to store all the information.

Another study by Mori and his team showed that having a website for driving schools can make things easier for students and more efficient for the school. Both of these studies say that using systems is a good idea, which is what BLRT Driving School wants to do.

Intelligent Scheduling and Instructor Matching Algorithms

Some researchers, like Thompson and Lewis think that having a computer system to schedule classes can help prevent mistakes and make sure instructors are available. They also think that having a system that can match instructors with the classes is a good idea. Garcia found that using algorithms can help prevent double-booking and make scheduling easier. These studies from countries show that having a smart system for scheduling is a good idea, which can help BLRT Driving School.

Artificial Intelligence and Chatbot Integration in Educational Services

Some researchers, like Schell and his team, think that using chatbots to answer questions can be very helpful. They found that chatbots can answer questions and reduce the workload of administrators. Pani and his team found that chatbots are good for answering questions but they should not replace human staff. These studies show that having a chatbot system can be very helpful which is something that BLRT Driving School can use.



Performance Analytics and System Acceptability

Finally, having a system to manage a driving school is very important. Some researchers, like Hernández-de-Menéndez and his team think that having a system to track data and provide analytics is very important. They found that having a dashboard can help administrators make decisions and improve the school. Davis and Chen found that having data visualizations can help administrators understand instructor workloads and student performance. These features can help BLRT Driving School make decisions and be sustainable.

In the Philippines a study by Mina found that using systems is better than manual systems because it is easier to use and more efficient. Another study by Cortez and Reyes found that using computers to manage schools can reduce mistakes and make it easier to keep track of documents. These studies show that having a system is very important which is something that BLRT Driving School needs.

On the local front, a study by Santos and Lim found that using computers to schedule classes can help prevent mistakes and make things easier for administrators. Another study by Bautista and his team found that using a system to assign instructors can improve the scheduling of driving classes. These studies show that using algorithms to schedule classes can be very helpful.

Locally, Villanueva found that using chatbots to answer questions can increase student engagement and enrollment. Another study by Atalaya and his team found that chatbots can help answer questions about fees and requirements and reduce downtime. These studies show that having a chatbot system can be very helpful for BLRT Driving School.



In the Philippines a study by Mendoza and Cruz found that having a system to track performance can help allocate resources and improve the school. Another study by Torres and Agbayani found that having a user- system is very important for the adoption of new technology. These studies show that having a system with analytics and a user-friendly interface is very important for BLRT Driving School.

In the Philippines, a recent study by Mayo (2022) on public schools in Cavite demonstrated that transitioning from conventional manual methods to a web-based enrollment system significantly enhances process reliability, administrative communication, and overall user satisfaction. In parallel, a local study evaluating an Automated Class Scheduling System at Bohol Island State University found that automated scheduling algorithms resolve the heavy administrative workload inherent in manual plotting by instantly detecting conflicts and optimizing instructor loading based on availability. These studies strongly support that integrating a centralized web-based enrollment system with an automated instructor-matching feature will eliminate operational errors, conserve manpower, and maximize institutional productivity for BLRT Driving School.



Chapter 2

METHODOLOGY DESIGN

This chapter presents the methodology and system design adopted in the development of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT. It outlines the Rapid Application Development (RAD) model, which was chosen to guide the planning, design, prototyping, and implementation of the system while emphasizing iterative design and active user involvement. The chapter details the four main phases of this approach:

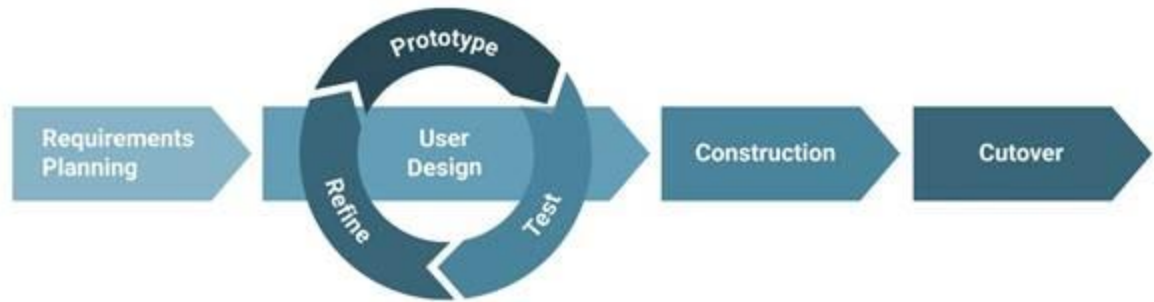
Requirements Planning, User Design, Rapid Construction, and Cutover. Additionally, it identifies the primary sources of data and the collaborative consultations used to establish system objectives and obtain relevant information from key stakeholders, including BLRT administrators, staff, driving instructors, and student applicants. Finally, this chapter describes the system architecture and enumerates the hardware, software, deployment, and user access requirements including tools such as Laravel, Livewire, MySQL, and Hostinger necessary to achieve a functional, efficient, and highly adaptive platform tailored for the driving school's operational environment.



System Development Methodology (SDM)

In software engineering, the use of a well-defined software development methodology (SDM) is crucial for ensuring a structured, efficient, and goal-oriented approach to system development. It serves as a framework that guides developers in systematically planning, designing, implementing, testing, and maintaining a system to meet user requirements effectively. For the proposed Web-Based Intelligent Driving School Management and Enrollment System for BLRT, selecting an appropriate SDM was a key consideration to align the project with the institution's specific administrative needs, operational constraints, and development timeframe.

This study adopted the Rapid Application Development (RAD) methodology due to its iterative nature and suitability for projects that demand quick development and continuous user feedback. RAD emphasizes short development cycles, early and repeated user involvement, and the rapid construction of prototypes. Given the manual and time consuming state of the current enrollment and scheduling processes at BLRT Driving School such as the reliance on physical whiteboards and manual tracking of student progress RAD provided the flexibility to develop working modules quickly. It allowed the researchers to gather immediate feedback from key stakeholders, including BLRT administrators, driving instructors, and student applicants, and adjust the system accordingly. Its iterative design enabled the development team to produce tangible system outputs early in the process, particularly for complex features like the automated instructor-matching algorithm, and continuously refine them based on user feedback, thereby ensuring the final platform closely matched the actual needs of its users.



Rapid Application Development (RAD)

RAD methodology is characterized by four key phases: Requirements Planning, User Design, Construction, and Cutover.

Requirements Planning – During the Requirements Planning phase, the developers gathered information about the current manual process by engaging with BLRT Driving School staff. The goal was to identify pain points, user expectations, and system requirements.

User Design – In the User Design phase, user interface mockups were created using design tools like Figma and reviewed in consultation with stakeholders. Feedback was incorporated to improve system flow and usability.

Construction – The Construction phase involved actual system coding using web technologies such as HTML, CSS, AlpineJS, TailwindCSS, Laravel, Livewire, PHP, and MySQL. This phase also included unit testing to ensure functionality aligned with specifications.



Cutover - In the Cutover phase, the system was deployed for pilot use, data was migrated, users were trained, and the project was evaluated for functionality and performance readiness.

The use of RAD ensured that the proposed system was not only built in a time-efficient manner but also with continuous stakeholder engagement ultimately leading to a solution that directly addresses the inefficiencies of the current manual setup in BLRT Driving School.



Source Data

The sources of data in this study are integral to understanding the operational challenges and user requirements related to managing student enrollments and scheduling at BLRT Driving School Inc. These sources include individuals and groups directly involved in or affected by the core operational processes such as enrollment, scheduling, instructor assignment, and performance monitoring.

Primarily, the Driving School Administration, including the school administrator, and staff, served as a major source of information. Through collaborative consultations with school administrator Mr. Jimmy L. Padilla and school staff insights were gathered regarding the inefficiencies of the current manual operations. Their input highlighted significant administrative burdens, such as manual booking causing double-booking issues, scheduling done by writing on physical boards, and the manual creation of certificates.

Secondly, the Driving Instructors, who are highly trained and accredited professionals conducting practical driving lessons, were consulted. Their experiences emphasized the difficulties in student progress monitoring, the manual tracking of student hours, the manual calculation of points, and the manual checking of instructor availability. They provided data supporting the need for accurate, real-time daily schedules and a digital platform to input student grades and remarks, replacing manual logbooks.

Thirdly, Student Applicants enrolling in Theoretical Driving Courses (TDC) and Practical Driving Courses (PDC) were considered a vital source of data. Their needs and challenges helped define the expectations for a streamlined enrollment process, real-time access to available schedules, and transparent tracking of course progress and payment status. Their input also highlighted the



need for faster response times to repetitive inquiries regarding rates and requirements, which justified the integration of a chatbot assistance module.

Lastly, Parents and Guardians acting as sponsors for student applicants, particularly minors, served as secondary sources. Their perspectives underlined the importance of payment transparency and the assurance of legitimate scheduling, allowing them to easily verify that students are attending their authorized 15-hour seminars and practical sessions.

In conclusion, the diverse sources of data identified in this study ranging from BLRT administrators and instructors to student end-users and their sponsors provided a comprehensive view of the existing challenges in the driving school's manual operations. By incorporating the insights, experiences, and expectations of these key stakeholders, the researchers were able to establish a comprehensive baseline of system objectives and operational constraints. This collaborative approach ensures that the proposed Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT will be responsive to real-world needs, thereby improving organizational efficiency, training quality, and overall service delivery.



Data Gathering Techniques

To ensure a comprehensive understanding of the current processes, user experiences, and system requirements at BLRT Driving School Inc. Multiple data gathering techniques were employed. These techniques were carefully selected and implemented to collect both qualitative and quantitative data from relevant sources.

Surveys and Questionnaires - One of the primary techniques used was the survey questionnaire. These were distributed to selected student applicants and administrative staff during the early stages of the Requirements Planning phase. The surveys were designed using structured questions to assess the frequency of scheduling conflicts, common repetitive inquiries regarding rates and requirements, and preferences for online enrollment and digital tracking. The data collected helped the development team identify recurring problems and prioritize system features such as the Chatbot Assistance Module and online scheduling that would directly address user needs.

Interviews - Another method applied was the interview technique, conducted with key personnel, specifically the BLRT administrator, Mr. Jimmy L. Padilla, and the professional driving instructors. These interviews were semi-structured, allowing participants to elaborate on their daily workflows and the administrative burdens they encounter. The interviews took place during the first phase of project development and provided in-depth qualitative data that supplemented the survey results. This approach enabled the developers to understand specific pain points, such as the difficulties in manual tracking of student hours, manual calculation of grades, and checking instructor availability.

Observation - Direct observation was also utilized to study how staff and instructors interact with the existing manual processes. The developers observed



the day-to-day operations, including how schedules are currently written on physical boards and how booking is manually managed. By witnessing the processes firsthand, the development team was able to document procedural gaps, user frustrations, and operational bottlenecks particularly the double-booking issues caused by manual entry which further validated the urgent need for system automation.

Document Analysis - In addition, document analysis was performed on physical enrollment forms, manual instructor logbooks, manually created certificates of completion, and official LTO course guidelines for the Theoretical Driving Course (TDC) and Practical Driving Course (PDC). This technique helped the researchers assess how records were currently being maintained, the consistency of the data, and the extent of manual documentation errors. The analysis also provided a clear benchmark for the type of data, such as student progress points that the system would need to digitize and manage.

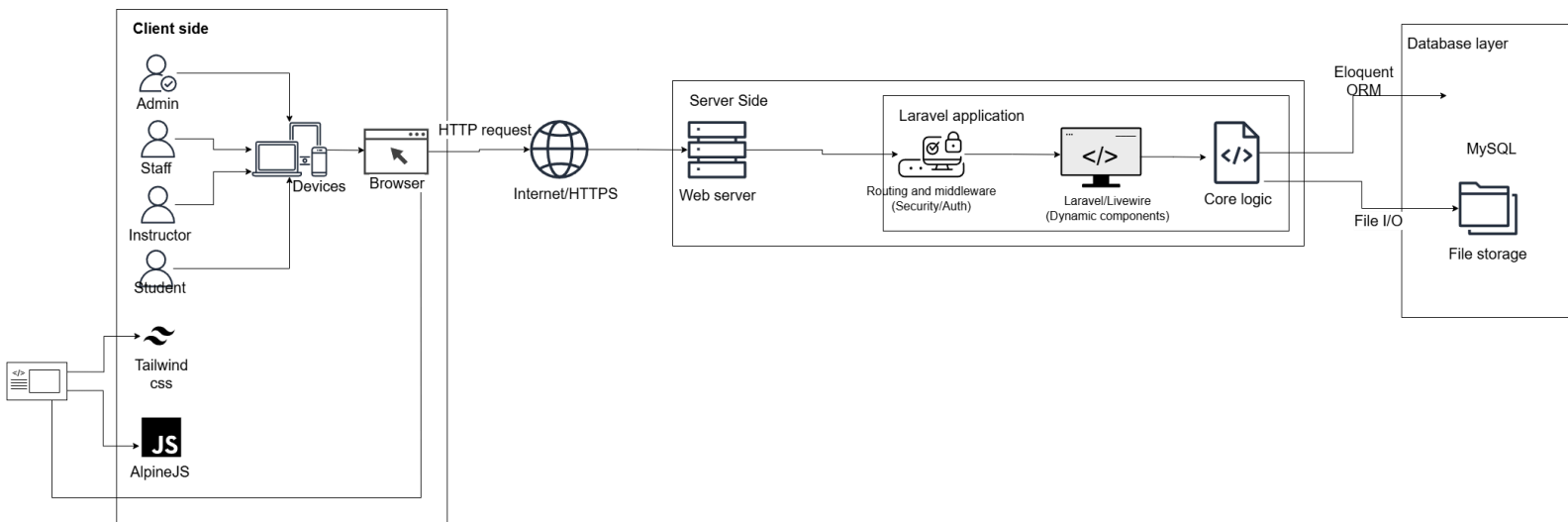
These data gathering techniques, applied during the initial planning and design stages, ensured that the development of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT was informed by real-world practices, grounded in user needs, and aligned with BLRT's institutional goals.



System Design

System Architecture Design

The structured blueprint that describes the parts, relationships, and data flow of a software system is called a system architecture design. It is essential in defining how users, technologies, and procedures are combined to accomplish the application's functional objectives. Efficiency, scalability, security, and maintainability are guaranteed by a well-designed system architecture, particularly in web-based solutions that support numerous users and roles. The system architecture design for the IT capstone project, Web-Based Intelligent Driving School Management and Enrolment System with Automated Instructor Matching and Performance Analytics for BLRT, clearly illustrates how users engage with the application, how data is processed and stored, and how services are provided via a web platform.



2.1 System Architecture



System architecture design is a foundational blueprint of a software system that includes its basic components and how users will be interacting with it, as well as how data will be flowing through it. The primary goal of a system architecture design is to define how users, technology, and processes integrate in order to achieve the functional and operational objectives of a software application. A well-designed system architecture is important in ensuring the efficiency, scalability, and security of a software application. For the IT Capstone Project, "Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT," it is evident that through the system architecture diagram, users will be able to have a clear and visualized understanding of how they will be interacting with it and how data will be processed and made available through the web application in a secure manner.

The diagram provided in this section describes the system architecture for the "Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT." This diagram indicates the main user roles for the proposed system, which are Student Applicants, Driving Instructors, System Administrators, and BLRT Administrative Staff. These user roles will be able to access the proposed system through a variety of client devices such as desktop computers, laptops, tablets, and mobile phones. These client devices will be able to access the proposed system via HTTP or HTTPS protocols over the internet, which is considered the medium of communication for these devices and the server.

At its heart lies the web server that acts as the application layer. It runs on top of the Laravel framework (PHP), along with Livewire and AlpineJS, which are in charge of handling all intricate business logic, real-time data processing, and dynamic user interaction (such as the intelligent instructor matching logic and



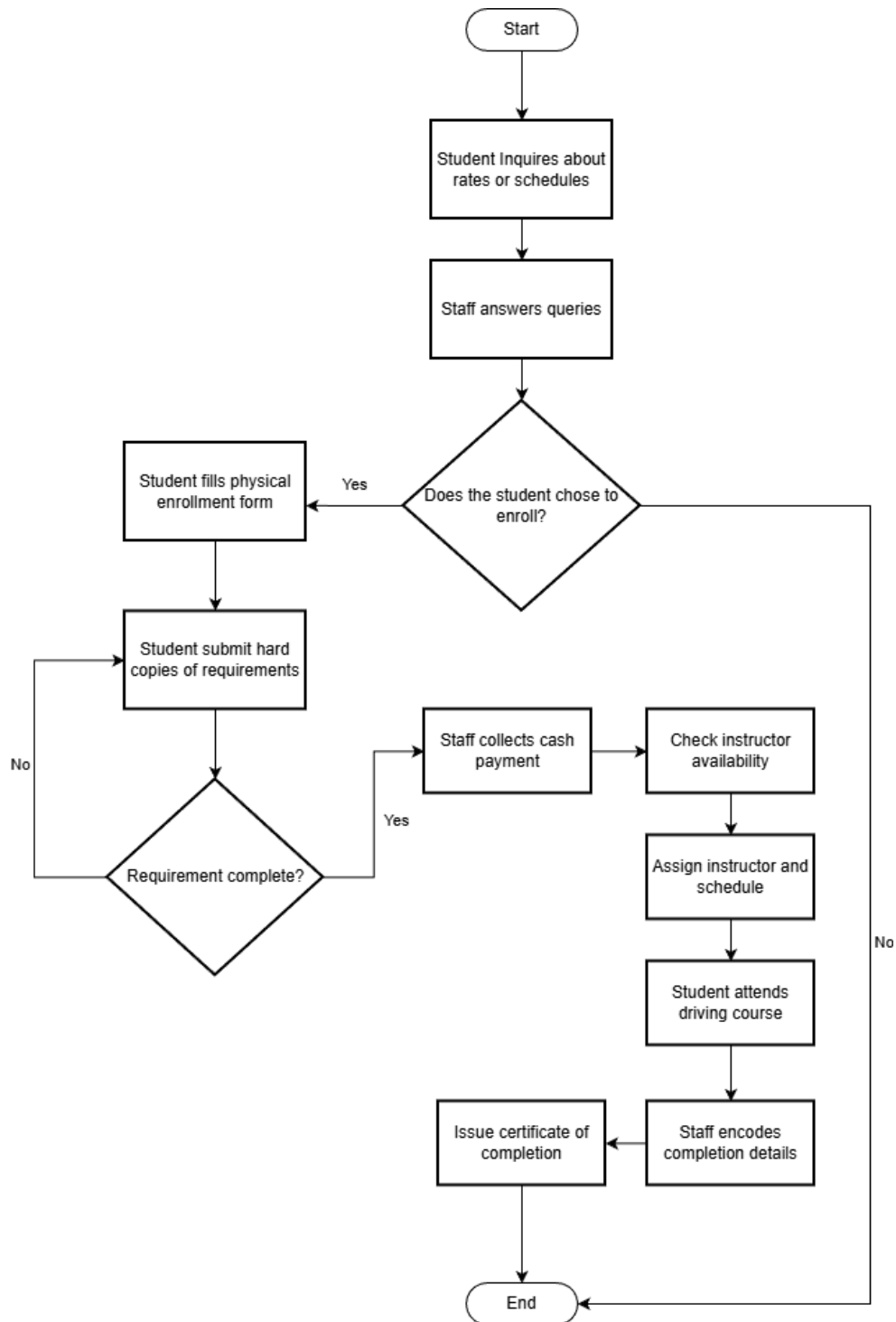
chatbot module). The server retrieves static frontend resources like HTML, TailwindCSS, and media files from the file system, whereas transactional data and application records are securely read from or written to a MySQL database. The SQL engine processes database queries to manage vital records like users, vehicles, courses, driving sessions, and analytics.

In conclusion, the system architecture design provides an overall framework for the development of the proposed system. It provides a secure system of access based on roles, allows for highly responsive web access from various devices, and facilitates the interaction of the front-end user interface and the back-end data services. It clearly defines the relationships between the user, devices, system components, and data repositories. Therefore, the development team is able to create a robust system that is strictly tailored to the needs of the operational processes of the BLRT Driving School Inc.



Existing Process Flowchart

A flowchart diagram serves as a visual representation that illustrates the step-by-step logic of a specific process using standardized symbols such as ovals, rectangles, diamonds, and arrows. Its primary purpose is to map out complex workflows in a clear and structured manner, allowing stakeholders to easily understand task sequences, identify critical decision points, and analyze inefficiencies—such as the manual scheduling conflicts and physical logbook redundancies currently observed at BLRT Driving School Inc.. In the development of the Web-Based Intelligent Driving School Management and Enrollment System, these diagrams are essential for documenting the intricate procedures related to online student enrollment, automated instructor matching, and practical course scheduling. By meticulously detailing these manual operations, the researchers establish the necessary groundwork for an automated, data-driven solution that directly addresses the challenges of student progress monitoring and administrative workload.





Entity Relationships Diagram (ERD)

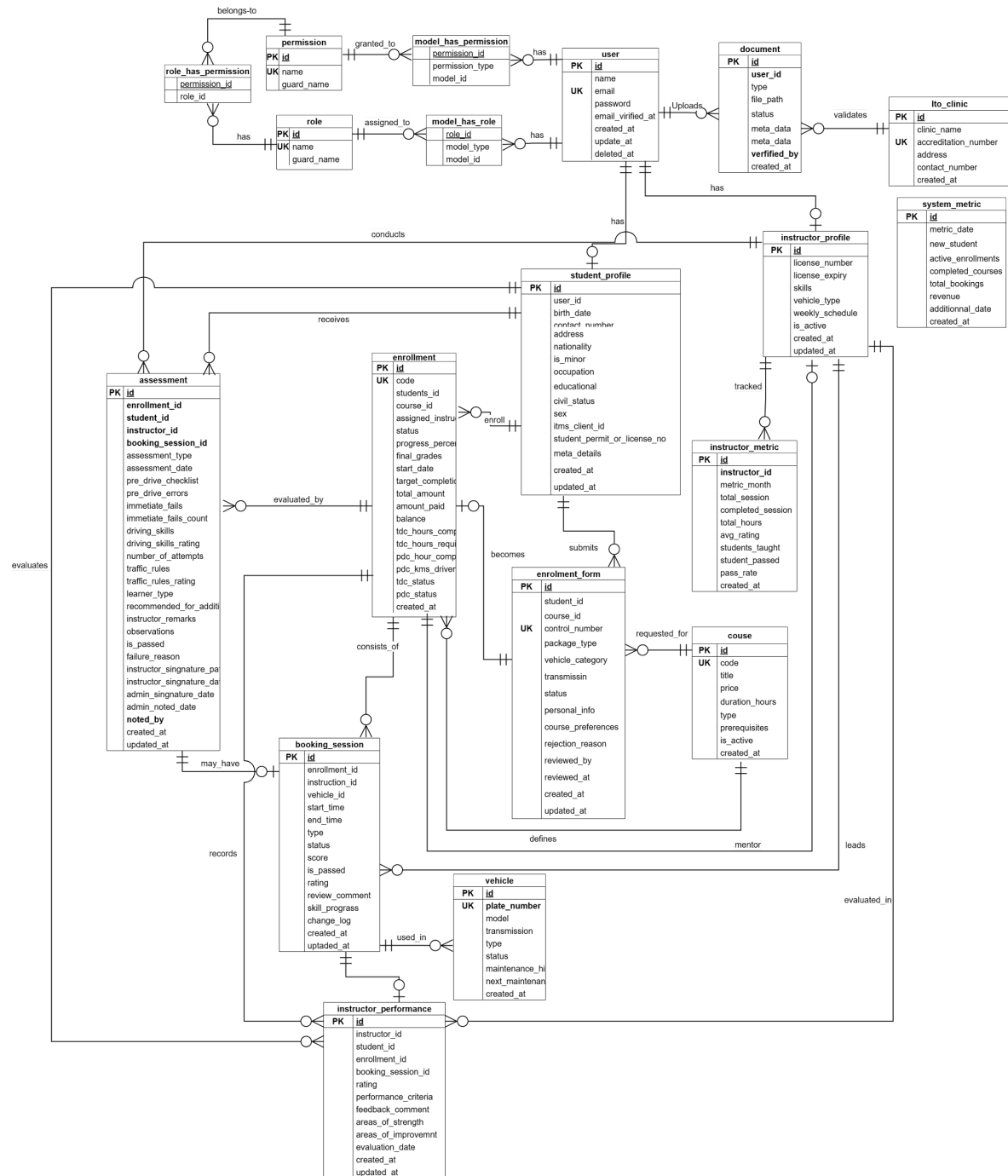


Figure 2.2: Entity Relationship Diagram



An Entity-Relationship Diagram (ERD) is a graphical representation of a system's data structure that illustrates the entities involved in the system, their attributes, and the relationships between them. It is an essential tool in database design, providing developers with a clear understanding of how data is stored, connected, and accessed within a system. In the context of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT, the ERD plays a crucial role in guiding the logical structure of the database. It ensures that all core entities such as students, driving instructors, course schedules (TDC and PDC), and performance assessment records are accurately represented and properly linked, thereby supporting the system's goal of streamlining online enrollment, automating instructor matching, and tracking student progress efficiently.

Presented in this section is the Entity-Relationship Diagram (ERD) that maps the database architecture of the proposed Web-Based Intelligent Driving School Management and Enrollment System for BLRT. This model illustrates the fundamental data entities—representing users, transactions, and system metrics—and the logical relationships connecting them. It serves as the data blueprint for how information is organized, stored, and retrieved to support the system's core functionalities.

At the foundation of the system's security and access control is the user entity. To manage authorization efficiently, the system utilizes a comprehensive role-based access control structure. The user entity connects to specific role and permission entities through junction tables such as `model_has_role`, `model_has_permissions`, and `role_has_permissions`. This design ensures that administrators, instructors, and students are securely granted the appropriate system privileges based on their designated roles.



Beyond basic access, the core user records branch out into specialized entities: `student_profile` and `instructor_profile`. These entities capture specific demographic and operational data relevant to each user type. Additionally, the system tracks regulatory compliance by allowing a user to upload a document. These uploaded documents are linked to an `lto_clinic` entity, representing the external medical or regulatory bodies responsible for validating the submitted requirements.

The core business process of student onboarding is captured through a clear enrollment pipeline. A `student_profile` initiates the process by submitting an `enrollment_form`, which is specifically requested for a chosen course. Once reviewed and approved, this form transitions into a formal enrollment record. This structured flow from an application to an active enrollment ensures accurate tracking of student intent and institutional acceptance.

To manage the practical driving lessons, an active enrollment consists of multiple `booking_session` records. The `booking_session` acts as a central transactional entity where various system elements intersect: it specifies the vehicle used for the lesson and establishes the relationship with the `instructor_profile` assigned to mentor the student. Furthermore, these sessions may have an assessment attached to them, allowing instructors to evaluate and record the student's driving competencies during the practical application.

Fulfilling the system's advanced analytical capabilities, several entities are dedicated to tracking evaluation and efficiency. While students are assessed on their driving, the system simultaneously evaluates the staff by recording `instructor_performance` linked to the booking sessions. These performance logs, alongside dedicated `instructor_metric` and `system_metric` entities, track the effectiveness, ratings, and workload of the driving instructors. This data acts as



the engine for the system's automated instructor matching algorithm and provides the administration with comprehensive performance analytics.

In summary, the ERD provides a holistic view of the driving school's data structure. By establishing clear relational links between secure user roles, structured enrollment pipelines, scheduling sessions, and rigorous performance metrics, the design ensures high data integrity. This robust foundation is vital for executing the complex automated matching and data-driven analytics required by the BLRT management system.



Data Flow Diagram (DFD)

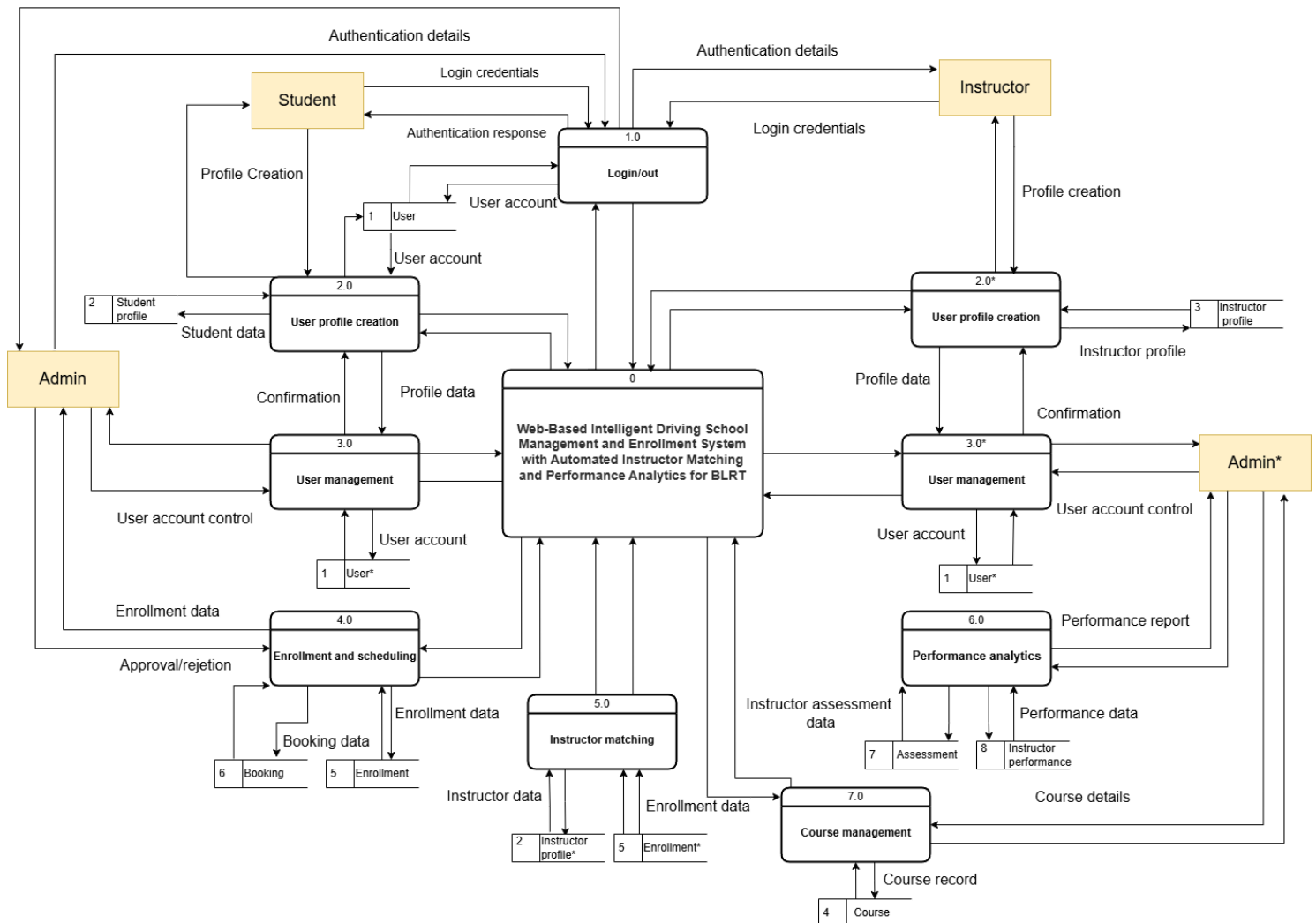


Figure 2.3: Data Flow Diagram



A Data Flow Diagram (DFD) is a high-level representation of a system's primary process and how it interacts with external entities and data stores. It is also referred to as a context diagram, as it outlines the boundaries of the system and the major data exchanges within and outside the system. Unlike detailed process flows, the Level 0 DFD provides a holistic overview of the system's functionality in a single, central process. For the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT, this diagram serves as a foundational visual aid that guides system developers, stakeholders, and future users in understanding the flow of data between the system and its external actors. It is particularly helpful in identifying user roles, system inputs and outputs, and the interaction with internal data repositories.

This section shows a Dataflow Diagram (DFD) which displays all key system interactions that the proposed system will operate through. The system which is identified as [0 Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT] establishes connections to three main external entities which are Student and Instructor and Admin together with its internal database systems. Data Flow Lines which show inbound and outbound user activity create relationships that describe how data moves through the system from submission to processing and storage until it reaches output.

The Data Flow Diagram (DFD) for Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT shows how the system connects to its main external systems which include students and instructors and system administrator and to its internal data storage systems that support system operation. The core model demonstrates how data flows into the system and



through specific Processes and out of the system which helps identify system limits and main operational processes.

The student serves as a key external entity that interacts with the system in several essential ways. Students begin their User profile creation process (2.0) by submitting their profile creation data and then receive confirmation of their profile creation. Students who complete their registration process can use the Login/out process (1.0) to log into the system by entering their login credentials and receiving authentication results. Students use the system to submit their enrollment information through the Enrollment and scheduling process (4.0) which they access after successful system entry. The system processes their schedules and bookings before it sends back to the student an approval or rejection result. The instructor serves as another main external entity who establishes contact with the system to perform his duties.



Implementation Plan

The implementation plan for the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT serves as a strategic roadmap to transition the project from development to a fully operational state. Utilizing the Rapid Application Development (RAD) methodology, this plan emphasizes iterative refinements and active stakeholder involvement to ensure the final system effectively addresses the manual scheduling and enrollment challenges faced by BLRT Driving School Inc..

A Gantt Chart is essential for visualizing the development schedule, illustrating the sequence and duration of tasks while identifying key personnel for each phase. In the context of this project, the timeline is structured around the four primary stages of the RAD model, ensuring a goal-oriented approach to system completion.

The project timeline is strategically structured over a sixteen-week period from February to May 2026, following the iterative phases of the RAD model to ensure rapid delivery and high alignment with institutional needs.



Task	Feb 2026				March 2026				April 2026				May 2026				Key personnel
	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	
Requirements planning																	Team B1 and stakeholder
User Design																	Team B1 and stakeholder
Rapid Construction (development)																	Team B1 Leader
Cutover (Deployment, testing, user training)																	Team B1 and system users

2.24 Project Timeline

The first phase, Requirements Planning, is carried out during the first week of February 2026. This early stage involves Team B1 and key stakeholders focusing on defining core functionalities such as enrollment automation and instructor matching to establish a technical blueprint tailored to the administrative needs of BLRT. The second phase, User Design, takes place during the second week of February. During this stage, the researchers collaborate with users to create and refine mockups for student interfaces and the analytics dashboard, ensuring usability through continuous feedback.

Rapid Construction, which forms the technical core of the project, is implemented from the third week of February through the end of April 2026. Led by the Team Leader, this phase involves iterative coding to build functional modules, including the intelligent matching algorithm and scheduling system, while integrating database and backend services. The final phase, Cutover, is scheduled for the first and second weeks of May 2026. This concluding task involves the full team and system users, focusing on finalizing the system,



conducting final testing, migrating data, and performing user training to ensure a smooth transition to the official launch.

Deployment Plan

The deployment plan outlines the final steps for the successful installation and launch of the system within the BLRT operational environment. Deployment is scheduled for the final weeks of the project timeline, marking the culmination of all development iterations. During this phase, the completed system will be installed on a cloud-hosted server via Hostinger Philippines with a proper domain configuration. System users, including school managers, driving instructors, and students, will participate in User Acceptance Testing (UAT) to verify that automated TDC/PDC requirements and instructor matching logic function correctly. Identified issues will be resolved promptly to ensure stability before the full launch. Following successful testing, structured training sessions will be conducted to equip BLRT staff with the knowledge needed to navigate the performance analytics and digital scheduling features effectively.

Resource Requirements

The successful implementation and long-term sustainability of the system depend on several critical hardware, software, and human components. Regarding Hardware Resources, development is supported by workstations including an AMD Athlon 3000G desktop and a Lenovo IdeaPad Slim 1 laptop, while end-users will require modern smartphones, laptops, or desktops to access the web portal. A stable internet or mobile data connection is essential as the system is web-based and requires real-time data synchronization.

Software Resources include the use of VS Code for development and Git for version control. The system itself is built on the TALL stack (TailwindCSS,



AlpineJS, Laravel, and Livewire) with a MySQL relational database management system. From a Human Resource perspective, the researchers of Team B1 are responsible for executing deployment and technical troubleshooting. The active participation of the BLRT administration, led by Jimmy L. Padilla, and the accredited driving instructors and staff are vital during the testing and training phases to ensure the system is ready for the daily operational demands of the driving school.



Chapter 3

RESULT AND DISCUSSION

This chapter presents the comprehensive results of the design, development, and evaluation of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT. It includes the system flowchart and data flow diagrams, which illustrate the logical structure and digital workflow of the application, as well as an overview of the developed modules and user interfaces designed to support students, driving instructors, and system administrators. The chapter also outlines the outcomes of the system testing and evaluation processes, which were conducted to ensure that the system met all functional, security, and performance standards prior to its official deployment at BLRT Driving School Inc. in San Carlos City.

These testing activities included functional testing to confirm that core features such as the automated enrollment, the intelligent instructor matching algorithm, and the digital grading system operated as intended; security testing to evaluate the system's role-based access control and the protection of student documentation; usability testing to assess the intuitiveness of the analytics dashboard and student portal; and user acceptance testing (UAT) to gather direct feedback from the BLRT staff and accredited instructors. Furthermore, this chapter presents the implementation results, detailing the deployment strategy on Hostinger Philippines, technical issues encountered during the iterative RAD phases, and the solutions implemented to optimize system responsiveness. The chapter concludes with a discussion of key findings that reflect the system's effectiveness in modernizing driving school operations, enhancing scheduling



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



accuracy, and promoting data-driven decision-making through automated performance analytics



Proposed System Flowchart

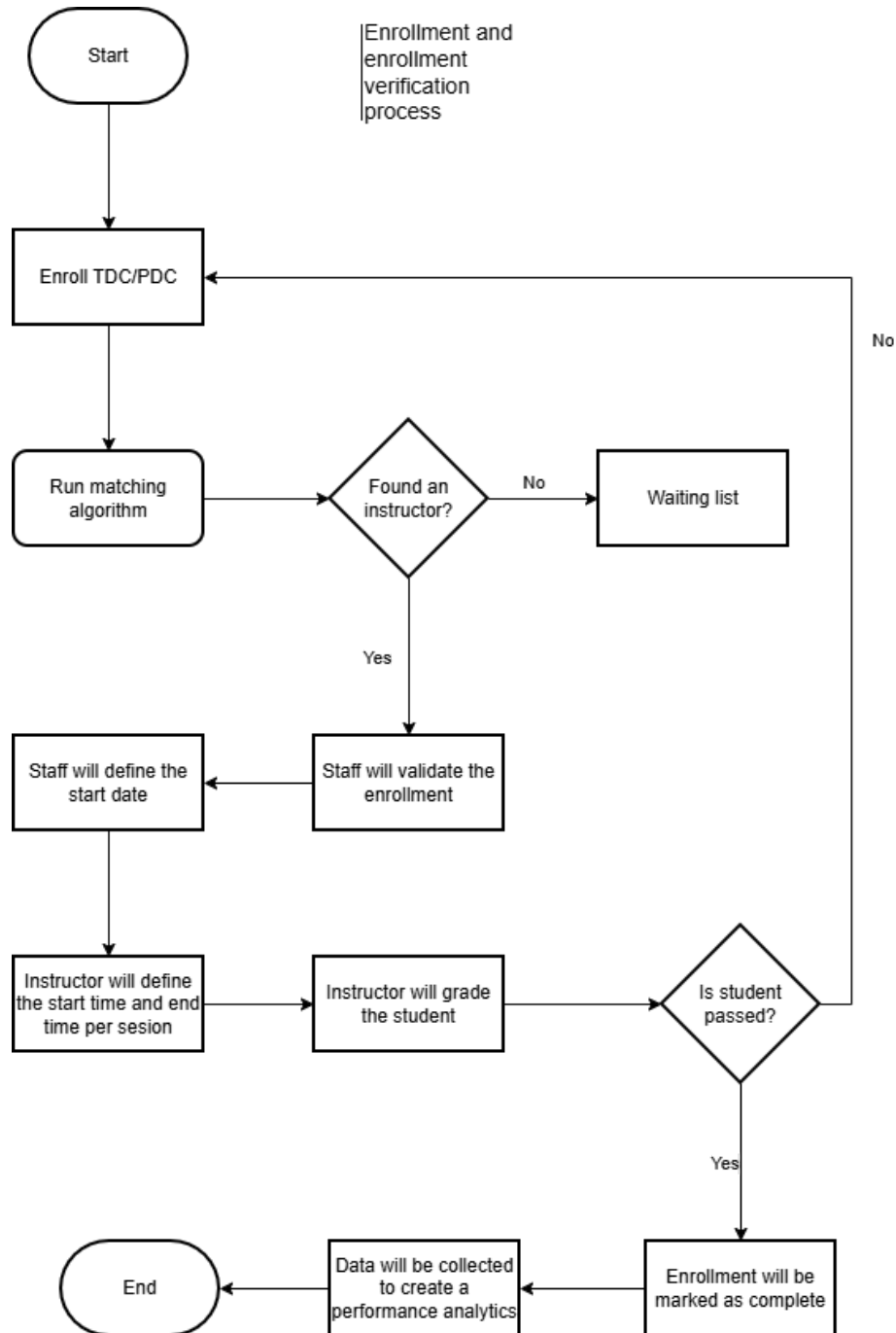


Figure 3.1: Enrollment and Verification Process

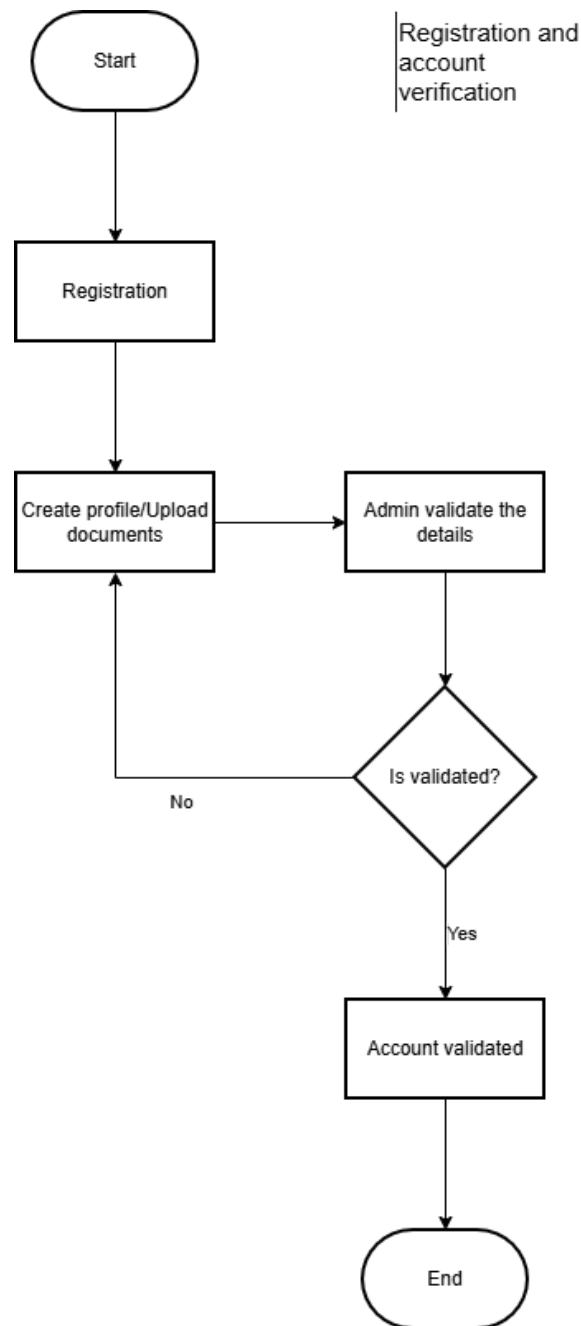
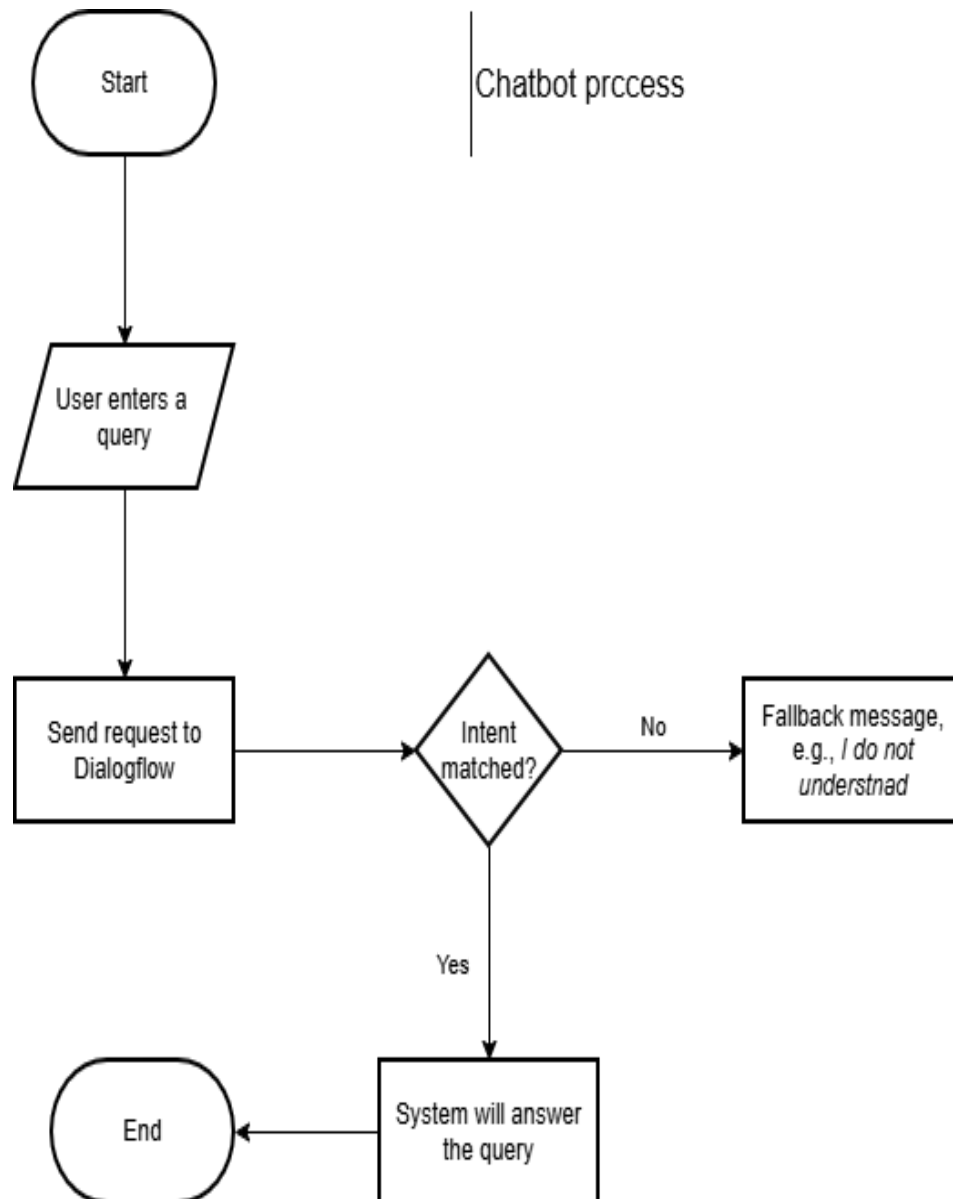


Figure 3.2: Registration and Verification



3.3 Chatbot Process



Student Enrollment and Course Progress (Staff) Flowchart

The student workflow begins when the user initiates a session by registering an account through the system portal. The student needs to establish a complete profile after registration and provide all required identification and permit documents for verification purposes. The system then awaits administrative validation; if the details are incorrect, the workflow loops back to the profile creation stage for correction. The student receives access to the enrollment module after completion of the verification process which allows them to register for the Theoretical Driving Course (TDC). The system executes an automatic matching process which connects the student to an available instructor. The student gets added to the waitlist when no instructor exists but after staff approval of the match the student starts their 15-hour seminar. The system evaluates student performance after the instructor completes their grading process. The student needs to re-enroll in the TDC if they fail; however, passing allows them to register for the Practical Driving Course (PDC). The system collects enrollment information through matching which tracks progress until the student workflow completes.



System Features and User Interface Discussion

This section details the core functionalities of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT based on the defined data flow processes. The platform is designed with a role-based architecture to ensure data security, operational efficiency, and a streamlined workflow. Access is strictly divided between authorized personnel specifically the System Administrator and the Staff.

Administrator

The Administrator holds the highest level of control within the system and is primarily responsible for governance, validation, and access control.

This feature enables the Administrator to review and verify newly registered student and instructor accounts. This includes validating submitted personal information and uploaded documents before granting system access.



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



The screenshot shows a web application interface for BLRT Driving School Inc. The main section is titled "Pending Registrations" with a subtitle "Thursday, May 7, 2026 - Review and manage applications". It features four summary cards: "Pending" (0 applicants), "Verified" (1 + 0 today), "Rejected" (0 + 0 today), and "Registration today" (0 applications). Below these is a filter bar showing "Pending (0)", "Verified (1)", and "Rejected (0)". A search bar is also present. The "Verified Instructors" section shows a table with one entry: John Instructor, License Number 38-4435-65778-1, Registration Date Apr 26, 2026 (1 week ago), and Status Verified.

Applicant	License Number	Registration Date	Status
John Instructor instructor@blrt.com	38-4435-65778-1	Apr 26, 2026 1 week ago	Verified

Figure 3.4: User Verification and Approval



This feature allows the Administrator to access, evaluate, and approve or reject uploaded requirements such as identification documents and enrollment prerequisites.

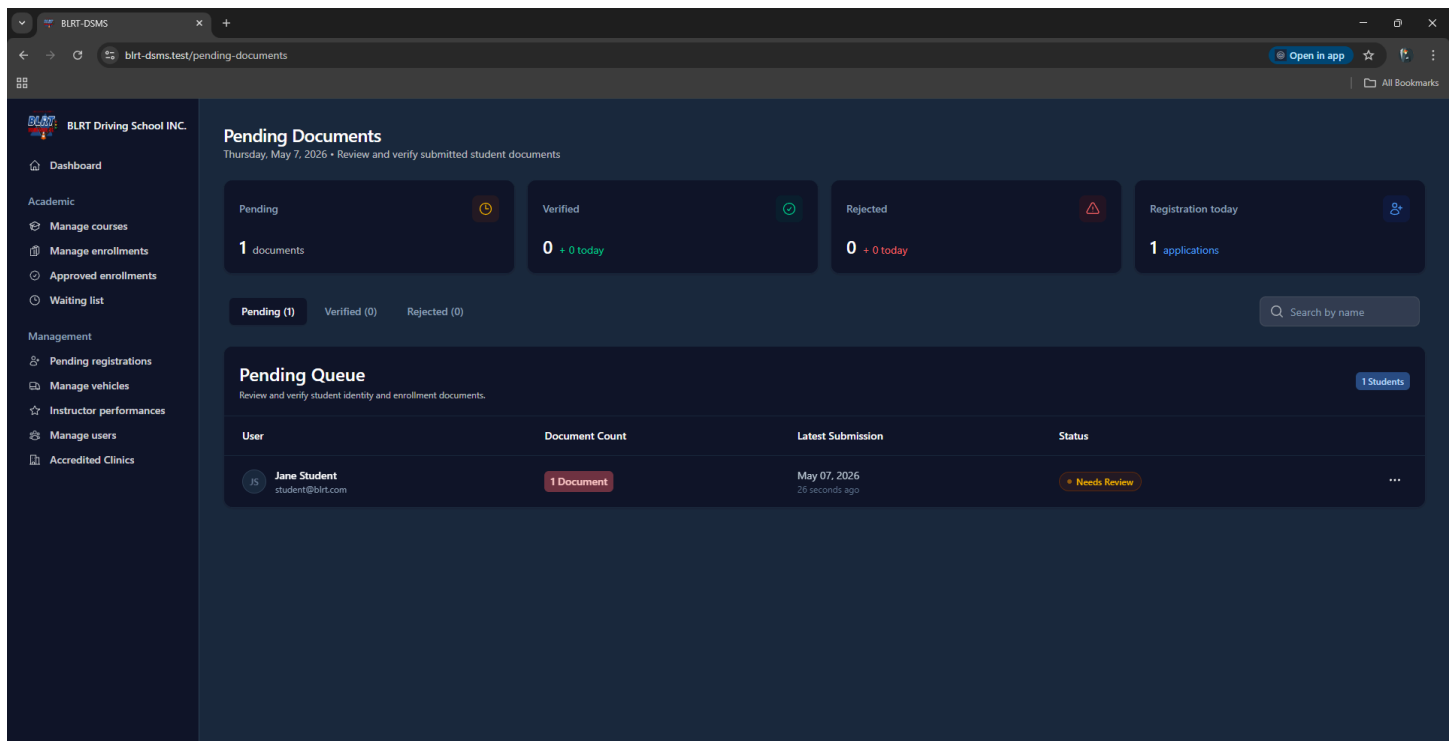


Figure 3.5: Document Review and Management



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



This feature provides functionality to monitor and validate student enrollments in both Theoretical Driving Course (TDC) and Practical Driving Course (PDC).

The screenshot displays the BLRT Driving School Inc. dashboard. The left sidebar contains navigation links: Dashboard, Academic (Manage courses, Manage enrollments, Approved enrollments, Waiting list), and Management (Pending registrations, Manage vehicles, Instructor performances, Manage users, Accredited Clinics). The main content area is titled 'Active Enrollments' with the subtitle 'Manage ongoing student courses and track their progress.' It features four summary cards: Total Records (3 total), Active (0 running), Pending Start (0 waiting), and Completed (1 finished). Below these is a filter bar with tabs: All (3), Active (0), Pending (0), Completed (1), and Dropped (0). A search bar is also present. The 'Enrollment List' section, titled 'View and manage all active and archived student enrollments.', shows 3 records in a table. The table has columns for Enrollment Code, Student, Course, Instructor, Status, Start Date, and Actions. The first row shows a completed enrollment for 'tdctstud' on 'Theoretical Driving Course THEORETICAL' by 'John Instructor'. The second and third rows show 'Waiting List' status for 'tdctest' and 'Jane Student' respectively, both on 'Theoretical Driving Course THEORETICAL' and 'Unassigned' instructors. A 'View Details' button is visible next to the second row.

Enrollment Code	Student	Course	Instructor	Status	Start Date	Actions
BLRT-C2END4UW	tdctstud tdctstud@blrt.com	Theoretical Driving Course THEORETICAL	John Instructor	Completed	Apr 26, 2026 1 week ago	...
BLRT-WF2ELZZN	tdctest tdctest@blrt.com	Theoretical Driving Course THEORETICAL	Unassigned	Waiting List	Apr 26, 2026 1 week ago	View Details Edit Details
BLRT-L6PHSKJN	Jane Student student@blrt.com	Theoretical Driving Course THEORETICAL	Unassigned	Waiting List	Apr 26, 2026 1 week ago	...

Figure 3.6: Enrollment Oversight



This feature enables the Administrator to assign, modify, or revoke user roles such as Student, Instructor, Staff, ensuring proper access control and system security.

The screenshot displays the 'Manage Enrollments' page of the BLRT-DSMS system. The interface includes a sidebar with navigation options like Dashboard, Academic, Management, and Accredited Clinics. The main content area shows a summary of enrollment statistics: 3 total enrollments, 0 pending review, 3 approved, and 0 rejected. Below this is an 'Enrollment Queue' table with columns for Control Number, Student, Course, Status, Submitted, and Actions. The table lists three entries, all with an 'Approved' status. A context menu is visible over the third entry, offering options to 'View Details', 'Move to Pending', and 'Reject Application'.

Control Number	Student	Course	Status	Submitted	Actions
BLRT-63U4ITX	tdctud tdctud@blrt.com	Theoretical Driving Course THEORETICAL	Approved	Apr 26, 2026 1 week ago	...
BLRT-6E6LP4TA	tdctest tdctest@blrt.com	Theoretical Driving Course THEORETICAL	Approved	Apr 26, 2026 1 week ago	View Details Move to Pending Reject Application
BLRT-HV8PA3ES	Jane Student student@blrt.com	Theoretical Driving Course THEORETICAL	Approved	Apr 26, 2026 1 week ago	...

Figure 3.7: Role-Based Access Control Management



This system feature allows the Administrator to manage courses, like creating, updating, and removing courses.

Create Course
Define the curriculum details, pricing, and requirements for a new driving module.

Course Title
e.g. Theoretical Driving Course

Type
Select...

Description
Briefly describe what the student will learn...

Price (PHP)
0.00

Duration (Hours)
15

Course Prerequisites
Determine the mandatory modules required before enrolling in this course.

☐ Theoretical Driving Course
BLRT-GIGFNT

Cancel **Save Course**

Course Directory
Manage your existing curriculum and pricing.

Search courses...

Code	Course Details	Price	Duration
BLRT-GIGFNT	Theoretical Driving Course Define the curriculum details, pricing...	33.00	2 hours

Figure 3.8: Course Management



The system feature allows the Administrator to oversee vehicle assets by performing operations such as creating, updating, and removing vehicle records, as well as managing and tracking scheduled maintenance dates for each unit.

Manage Vehicles
Register and track the university's driving school fleet.

Vehicle Model: Plate Number:
Transmission: Vehicle Type:
[Cancel](#) [Save Vehicle](#)

Vehicle Inventory
View and manage all registered vehicles.

Model	Plate Number	Type & Trans.	Status	Next Maintenance
asdf	SPG 883	automobile • Auto	Available In Use Maintenance	N/A
Honda	SPG 845	motorcycle • Auto	Available In Use Maintenance	N/A
Toyota Vios	SPG 883	automobile • Manual	Available In Use Maintenance	N/A
Rusi	SPG 888	motorcycle • Manual	Available In Use Maintenance	Apr 28, 2026
Honda Civic	SPG 855	automobile • Auto	Available In Use Maintenance	Apr 27, 2026

Figure 3.9: Vehicle Management



School Administrator

The School Administrator utilizes the system primarily for monitoring performance and making strategic decisions. The system provides access to visualized reports and summaries, including student enrollment trends, instructor workload, and overall operational performance.

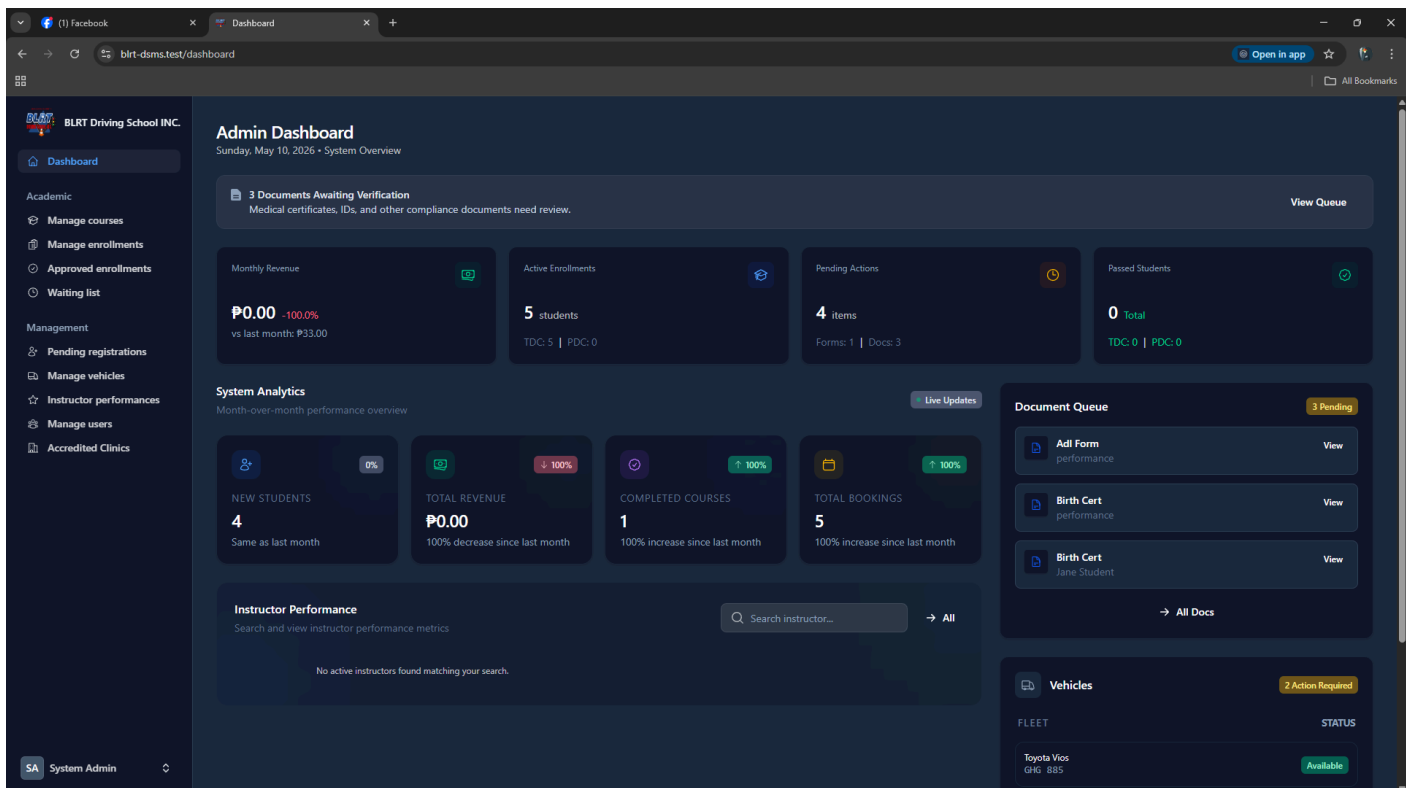


Figure 3.10: Performance Analytics Dashboard



Staff

The Staff is responsible for handling day-to-day administrative operations within the driving school.

This feature allows staff to review and process student enrollments, ensuring that all requirements are complete and accurate.

The screenshot displays the 'Manage Enrollments' interface of the BLRT-DSMS system. The dashboard includes a sidebar with navigation options like 'Dashboard', 'Enrollments', 'Manage enrollments', 'Approved enrollments', and 'Waiting list'. The main content area shows a summary of enrollment statistics: 3 total enrollments, 0 pending review, 3 approved, and 0 rejected. Below this is an 'Enrollment Queue' table with 3 records, listing student details, control numbers, courses, and submission dates.

Control Number	Student	Course	Status	Submitted	Actions
BLRT-63UJ4ITX	tdcstud tdcstud@blrt.com	Theoretical Driving Course THEORETICAL	Approved	Apr 26, 2026 1 week ago	...
BLRT-GE6LP4TA	tdctest tedctest@blrt.com	Theoretical Driving Course THEORETICAL	Approved	Apr 26, 2026 1 week ago	...
BLRT-HMHPA3ES	Jane Student student@blrt.com	Theoretical Driving Course THEORETICAL	Approved	Apr 26, 2026 1 week ago	...

Figure 3.11: Enrollment Validation



This feature provides tools to oversee and manage the master schedule of training sessions, ensuring proper allocation of instructors, vehicles, and time slots.

The screenshot displays the BLRT-DSMS web application interface. The left sidebar contains navigation links: Dashboard, Academic, Manage enrollments, Approved enrollments (selected), and Waiting list. The main content area is titled 'Active Enrollments' with the subtitle 'Manage ongoing student courses and track their progress.' It features four summary cards: Total Records (3 total), Active (0 running), Pending Start (0 waiting), and Completed (1 finished). Below these cards is a filter bar with tabs for All (3), Active (0), Pending (0), Completed (1), and Dropped (0). A search bar is also present. The 'Enrollment List' section shows a table of active enrollments with columns for Enrollment Code, Student, Course, Instructor, Status, Start Date, and Actions. The table contains three records, with the first two having 'Completed' and 'Waiting List' status respectively. The third record is also in 'Waiting List' status. A 'View Details' button is visible next to the second record.

Enrollment Code	Student	Course	Instructor	Status	Start Date	Actions
BLRT-C2END4UW	tdcstud tdcstud@blrt.com	Theoretical Driving Course THEORETICAL	John Instructor	Completed	Apr 26, 2026 1 week ago	...
BLRT-WF2ELZZN	tdctest tdctest@blrt.com	Theoretical Driving Course THEORETICAL	Unassigned	Waiting List	Apr 26, 2026 1 week ago	View Details Edit Details
BLRT-L6PHSKJN	Jane Student student@blrt.com	Theoretical Driving Course THEORETICAL	Unassigned	Waiting List	Apr 26, 2026 1 week ago	...

Figure 3.12: Schedule Management



This system feature facilitates coordination between students and instructors by managing assignments and ensuring schedule consistency.

The screenshot displays the 'Student Waiting List' interface of the BLRT DSMS. The interface is dark-themed and includes a sidebar with navigation options: Dashboard, Academic, Manage enrollments, Approved enrollments, and Waiting list. The main content area shows a summary of the waiting list with four cards: Total in Waiting List (2 students), Practical Requests (0 active), Theoretical (2 pending), and Longest Wait (11 days). Below this, there is a section titled 'Manual Assignment Required' with a search bar and a table of students. The table has columns for Enrollment Code, Student, Course Details, Wait Duration, and Action. Two students are listed: 'tdctest' and 'Jane Student', both with a wait duration of 11 days. Each student has an 'Assign Instructor' button.

Enrollment Code	Student	Course Details	Wait Duration	Action
BLRT-WF2ELZZN	tdctest tedctest@blrt.com	Theoretical Driving Course THEORETICAL	11 Days 1 week ago	Assign Instructor
BLRT-LGPHSKJN	JS Jane Student student@blrt.com	Theoretical Driving Course THEORETICAL	11 Days 1 week ago	Assign Instructor

Figure 3.13: Instructor and Student Coordination



Driving Instructors

Driving Instructors interact with the system to manage their teaching responsibilities and monitor student performance.

This system feature allows instructors to access their assigned schedules, including session dates, times, and student assignments.

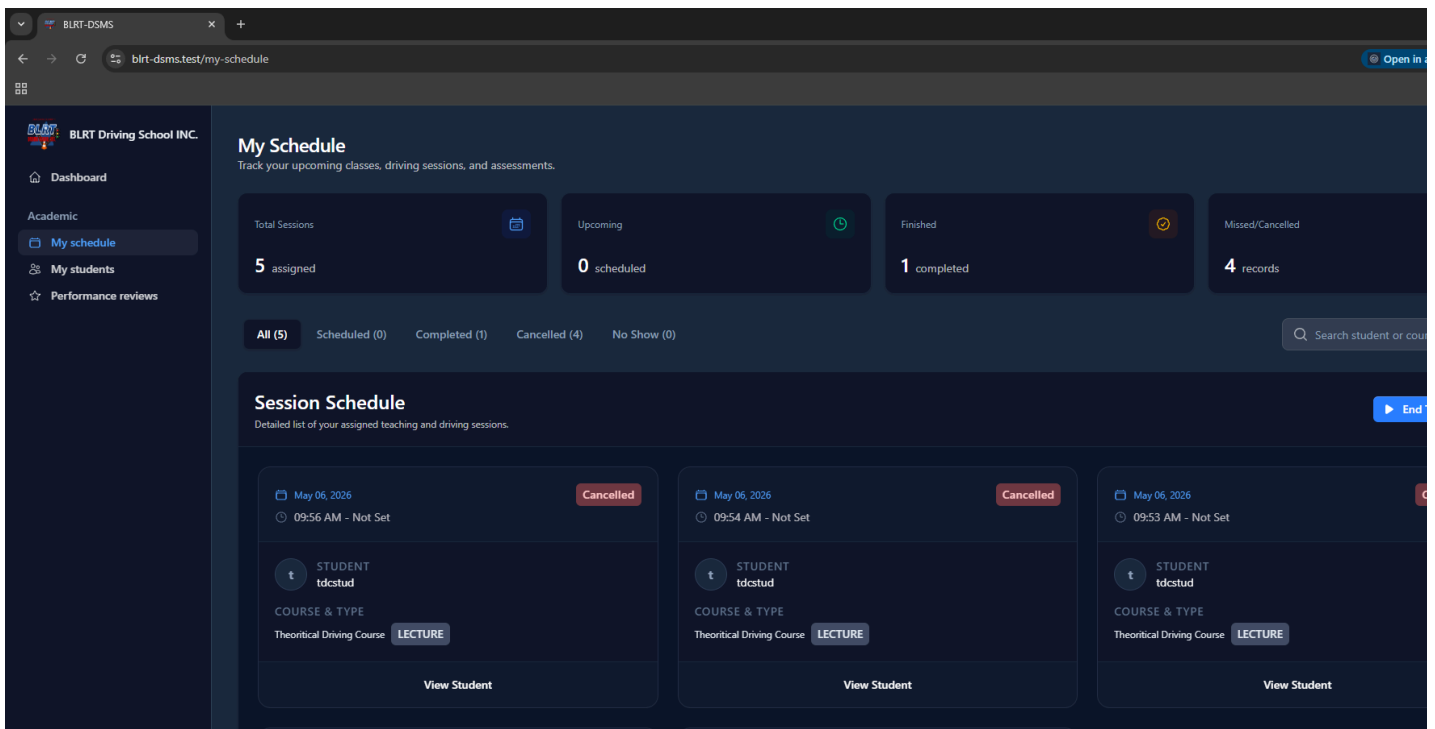


Figure 3.14: Schedule Viewing and Management



This feature enables instructors to track and update student progress in both TDC and PDC, including completed hours and training milestones.

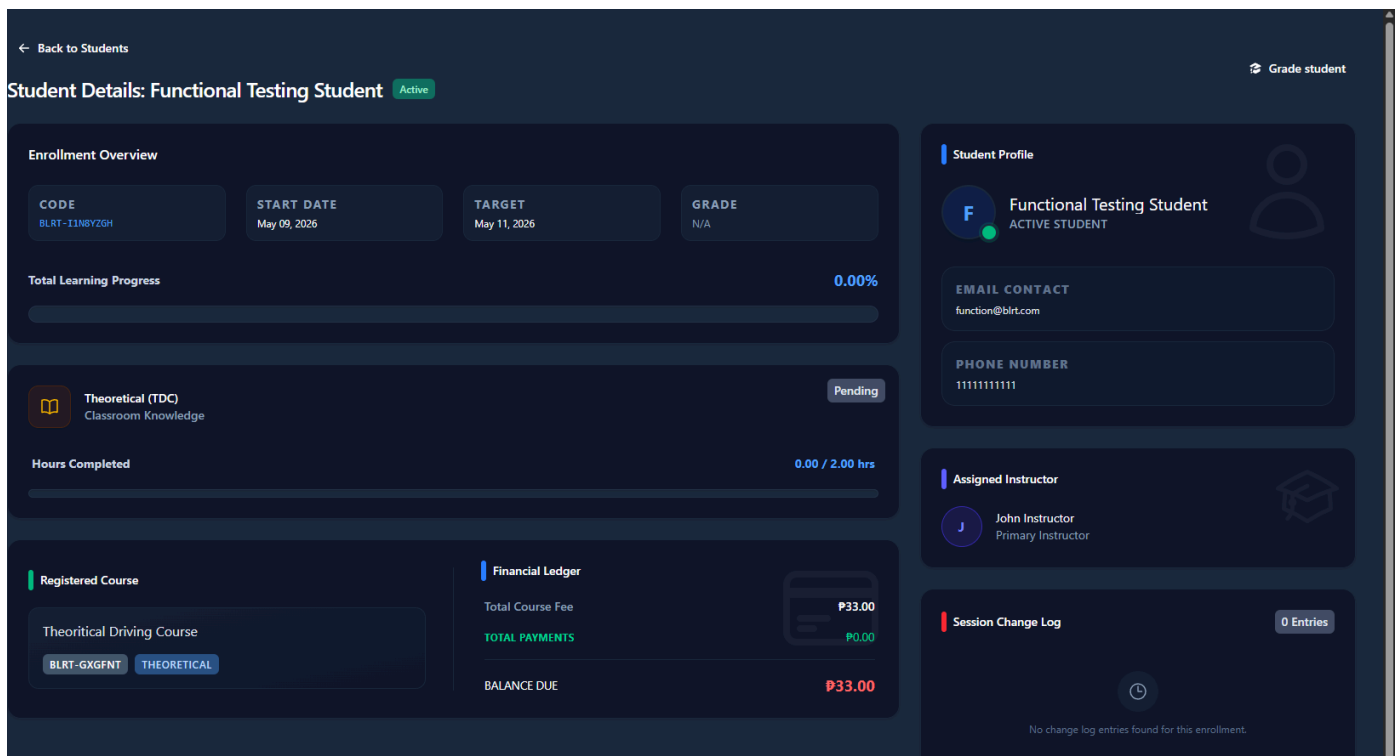
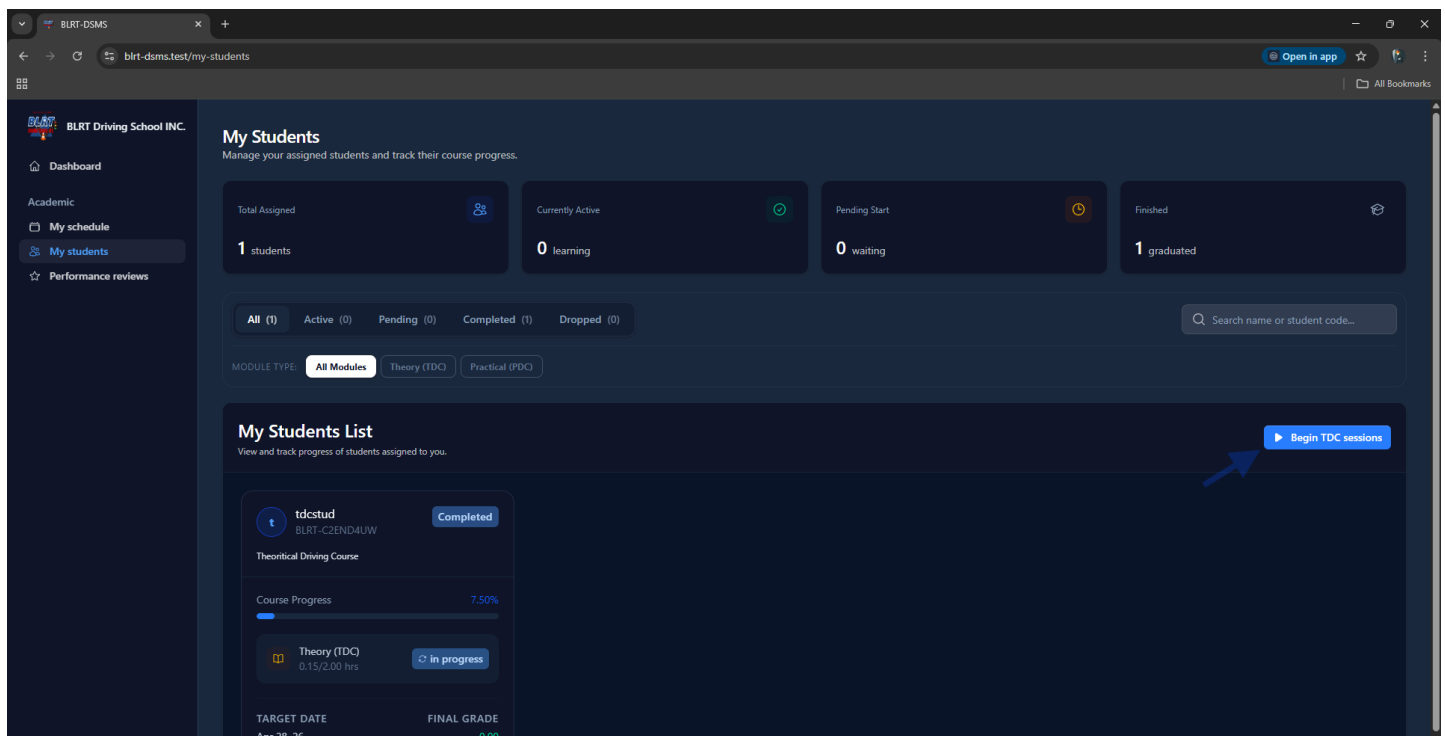


Figure 3.15: Student Progress Monitoring



This system feature provides functionality for recording student attendance during training sessions.



3.16 Figure 13. Attendance Recording



The system feature allows instructors to input grades, remarks, and evaluations based on student performance during practical sessions.

The screenshot shows a modal window titled "Update Official Grade" with a close button in the top right corner. The subtitle reads "Provide or update the final evaluation result for **tdcstud.**".

The form contains the following sections:

- Numerical Score (%)**: A slider input field currently set to 0.00, with a bar chart icon on the left and a vertical double-headed arrow on the right.
- Final Evaluation Result**: Two radio button options. The "PASSED" option is selected, indicated by a green checkmark icon. The "FAILED" option is unselected, indicated by a red 'X' icon.
- Instructor Remarks & Observations**: A text area with a placeholder text: "Detailed feedback on driving performance, technical skills, or specific areas for improvement...".

At the bottom of the modal, there are two buttons: "Cancel" and "Submit Grade".

Figure 3.17: Performance Evaluation and Grading



Student / Learner

The Student serves as the primary end-user of the system, interacting with various features related to enrollment and learning progress.

This feature enables students to create accounts, submit required documents, and enroll in available driving courses (TDC and PDC).

Course Options
Configure additional options for your course

Refresher Course
Check this if you are actively taking this course as a refresher. ☐

Emergency Contact
Who should we contact in case of an emergency?

Contact Person Name
Full Name

Contact Number
+63

Course Preferences
Help us tailor your training schedule and experience

Preferred Training Days

☐ Monday	☐ Tuesday	☐ Wednesday	☐ Thursday
☐ Friday	☐ Saturday	☐ Sunday	

Rules and Regulations for Students
Please read and acknowledge the following terms

- 01 Tuition Fee shall be fully Paid on the first day of driving lesson.
- 02 Tuition Fees for TDC/PDC are Non-Refundable once paid.
- 03 Students who withdraw or wish to withdraw from the school, for any reason, are not entitled to a refund of any fees paid.
- 04 Enrollment Fees is non Transferable.
- 05 No food, smoking or cellphone during lesson proper.
- 06 Cancellation of Practical Driving shall be within 1 day before schedule, penalty of 150 pesos if no notice of cancellation.
- 07 Student under influence of liquor/drugs will not be given driving lesson.
- 08 Strictly NO WEARING OF SHORT PANTS, SANDOS, SUPPERS.

Figure 3.18: Online Registration and Enrollment



This system feature allows students to upload necessary requirements for verification, such as identification and application forms.

Facebook x BLRT-DSMS x +

blrt-dsms.test/student/upload-document

Open in app ☆ All Bookmarks

BLRT Driving School INC.

Dashboard

Academic

My schedule

Academic records

studentPerformance

Upload Documents

Please provide the required documents to proceed with your enrollment.

Document Type

Birth Certificate

Select document type...

Birth Certificate

Medical Certificate

ADL Form

Valid ID

Marriage Contract

TDC Certificate

TIN ID

Choose file No file chosen

Submit Document

Figure 3.19: Document Submission



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



This feature provides students with access to available schedules, allowing them to view assigned sessions and training timelines.

The screenshot displays the 'My Schedule' page for BLRT Driving School INC. The page is divided into several sections:

- Header:** BLRT Driving School INC. logo and navigation links (Dashboard, Academic, My schedule, Academic records).
- My Schedule:** Track your upcoming theoretical and practical training sessions. Includes a 'Back to Dashboard' link.
- Theoretical Driving Course:** Enrollment Code: BLRT-VS1E8700. Status: Active. Includes a table with columns: START DATE (May 09, 2026), TARGET END (May 11, 2026), TDC PROGR (0.00 / 2.00 h), and PAYMENT (33.00 / 33.00). Overall Progress: 0%. Assigned Instructor: John Instructor.
- Module Status:** TDC Module (Not Started), PDC Module (Not Started).
- Training Itinerary:** Your detailed session plan chronologically ordered. 1 Sessions Total. Today's Session: MAY 10 SUN, LECTURE SESSION, 9:27 AM - TBA, Scheduled, John Instructor.
- Footer:** studentPerformance link.

Figure 3.20: Course Scheduling Access



This system automatically assigns suitable instructors based on availability, specialization, and course requirements.

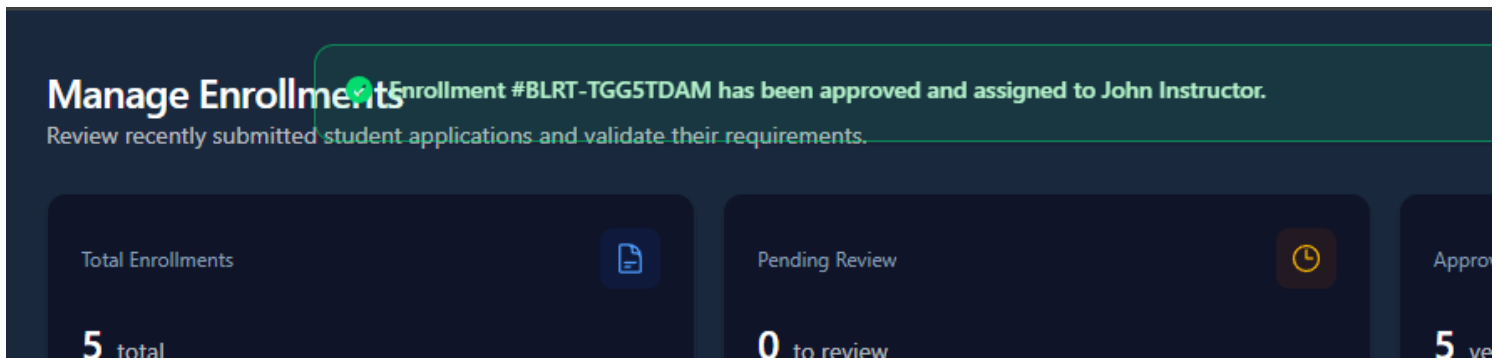


Figure 3.21: Intelligent Instructor Matching



This system feature enables students to monitor their training progress, including completed sessions, attendance, and performance records.

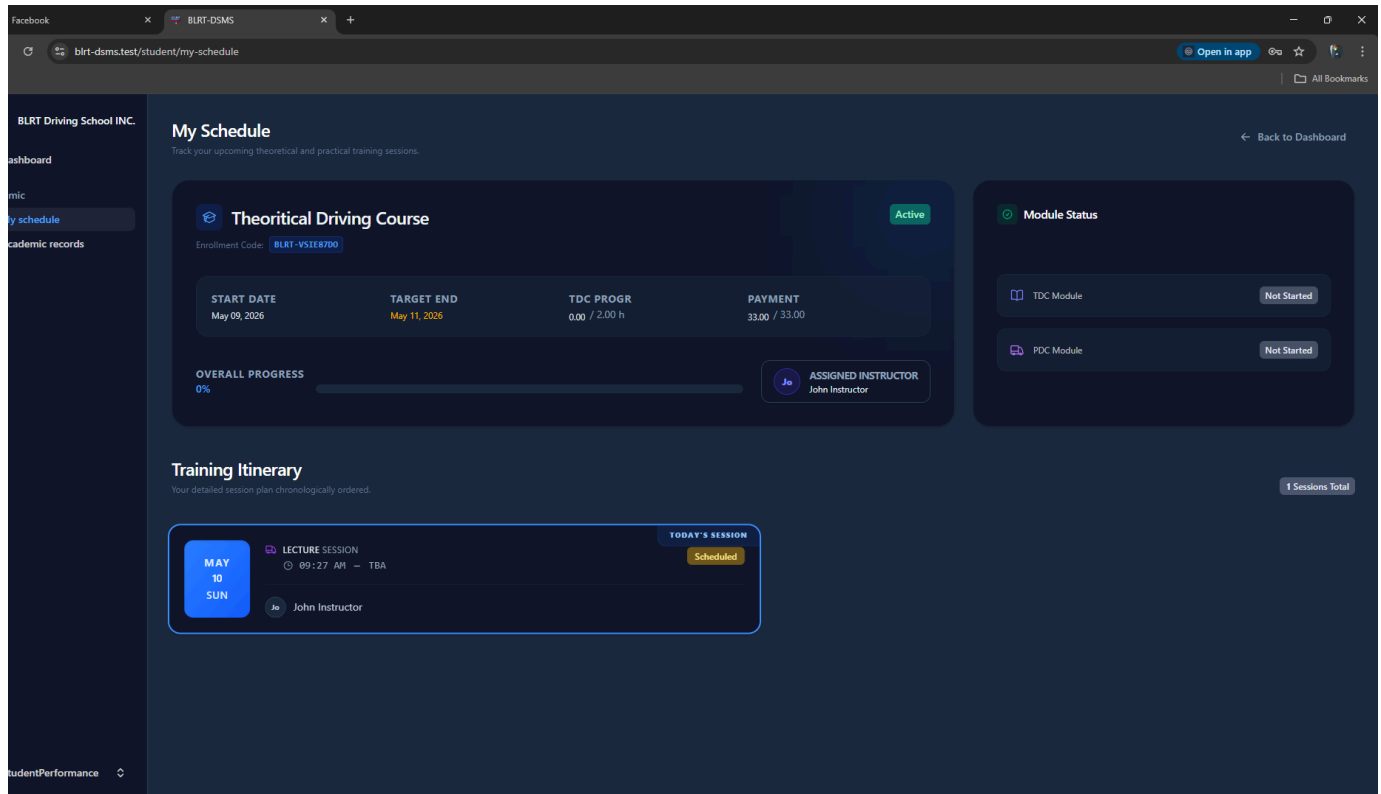


Figure 3.22: Progress Tracking



This system feature includes a chatbot module that provides automated responses to frequently asked questions regarding enrollment procedures, requirements, and fees.

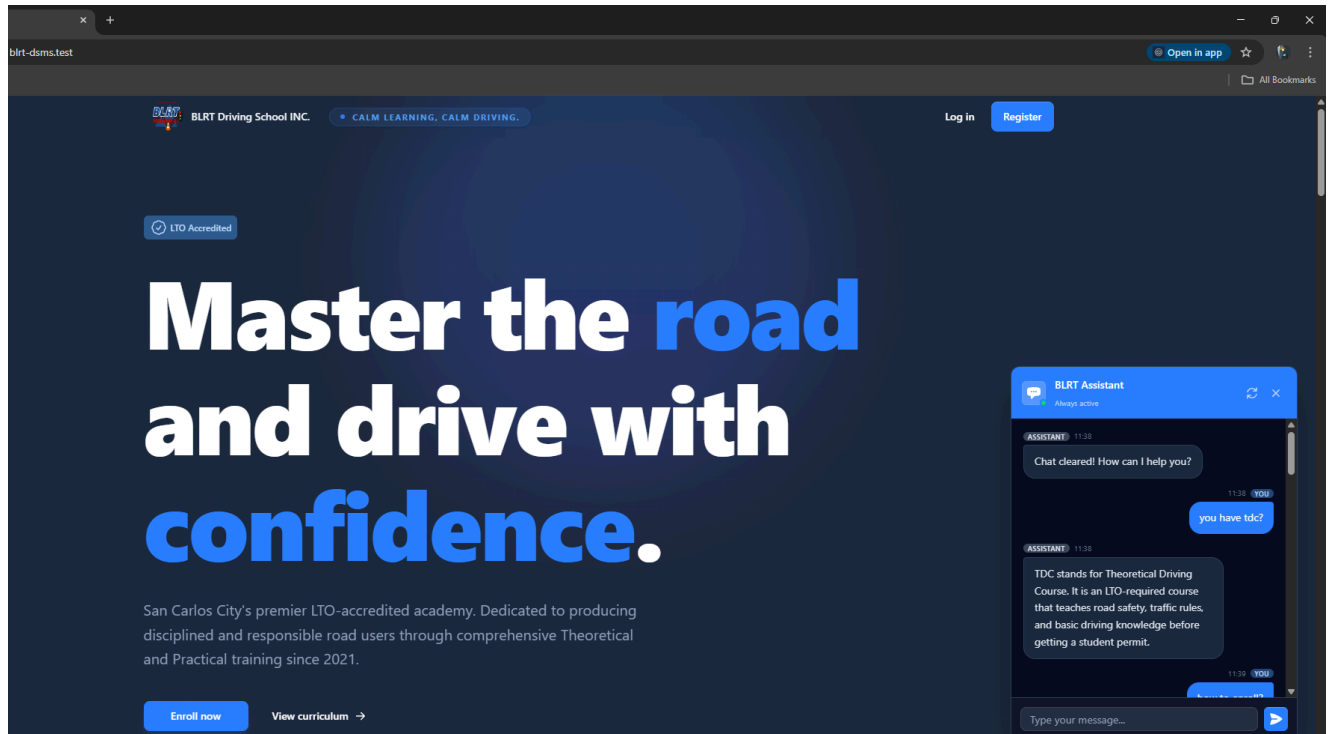


Figure 3.23: Chatbot Assistance



System Testing and Evaluation

The System Testing and Evaluation phase is a crucial component of the Software Development Life Cycle (SDLC), aimed at ensuring that the proposed Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT Driving School operates effectively, securely, and efficiently. This phase focuses on validating that the system meets its intended objectives, including automation of enrollment, scheduling, instructor assignment, and performance monitoring. The evaluation process is designed to assess the system's functionality, performance, security, usability, and overall acceptability among its intended users, namely the Super Administrator, School Manager, Staff, Driving Instructors, and Students. At this stage, the testing plan outlines the methodologies and approaches to be used in verifying system quality; however, actual testing results are not yet included.

Functional Testing

Functional Testing is conducted to verify that all system features operate according to the specified requirements and intended behavior. This includes validating core modules such as user registration and authentication, enrollment processing, scheduling, instructor matching, student monitoring, and certificate generation. The testing plan involves the creation of detailed test cases for each functionality, specifying inputs, processes, and expected outputs. For example, test scenarios will include successful student registration, accurate enrollment in TDC and PDC courses, correct instructor assignment based on availability, and proper generation of training certificates. Manual testing will be performed by simulating real user interactions across different roles. Each test case will be documented to determine whether the system produces the expected results or



exhibits any functional errors. This ensures that all modules are working cohesively and reliably prior to deployment.

Security Testing

Security Testing aims to evaluate the system's ability to protect sensitive data and prevent unauthorized access. Given that the system handles personal information, training records, and role-based access control, ensuring data security is a top priority. The testing plan includes validating authentication mechanisms, enforcing role-based access restrictions, and ensuring proper data validation. Specific test scenarios will involve attempts to access restricted pages without authorization, submission of invalid or malicious inputs, and testing for common vulnerabilities such as SQL injection and cross-site scripting (XSS). The expected outcome is that the system effectively denies unauthorized access, sanitizes user inputs, and protects sensitive data from exposure. These tests will be conducted manually and, where applicable, supported by automated tools to identify potential vulnerabilities.

Usability Testing

Usability Testing is designed to evaluate the system's ease of use, accessibility, and overall user experience for all intended users. This is particularly important as the system will be used by individuals with varying levels of technical expertise. The testing plan involves the development of a structured usability evaluation instrument, such as a survey questionnaire utilizing a Likert scale. Participants, including students, instructors, and administrative staff, will be asked to perform key tasks such as registration, enrollment, schedule viewing, and progress tracking. After completing these tasks, users will provide feedback on system navigation, interface clarity,



responsiveness, and overall satisfaction. Observations will also be recorded to identify any difficulties encountered during system interaction. This approach ensures that the system is not only functional but also user-friendly and intuitive.

User Acceptance Testing (UAT)

User Acceptance Testing (UAT) serves as the final validation phase to determine whether the system meets the expectations and requirements of its stakeholders. It ensures that the developed system is suitable for actual operational use within BLRT Driving School. The testing plan includes defining real-world scenarios that reflect typical system usage, such as enrolling students, assigning instructors, managing schedules, and generating reports. Selected end-users, including administrators, staff, and instructors, will be invited to test the system using these scenarios. Participants will evaluate the system based on criteria such as functionality, reliability, efficiency, and overall satisfaction. Feedback will be collected through structured evaluation forms and interviews. Any identified issues or suggested improvements will be documented and addressed prior to final deployment.

Performance Testing

Performance Testing is conducted to assess the system's responsiveness, stability, and efficiency under varying workloads. This ensures that the system can handle normal and peak usage conditions without significant delays or failures. The testing plan involves simulating multiple users accessing the system simultaneously, particularly during high-demand periods such as enrollment and scheduling. Key performance indicators include page load time, response time for data retrieval, and system stability during concurrent operations.



Test scenarios will include bulk student registrations, simultaneous schedule access, instructor assignment processing, and report generation. The expected outcome is that the system maintains acceptable performance levels without crashes or significant slowdowns.



Implementation Results

The implementation of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT was conducted through a structured pilot deployment within BLRT Driving School Inc. During the final phase of development, the system was deployed in a controlled test environment using the available hardware and internet connection of the driving school. This pilot testing phase enabled the researchers to observe the system's actual performance, validate database integrity, and gather feedback from administrators, staff, instructors, and students who interacted with the platform.

As part of the implementation process, functional testing was conducted to verify that all core features operated according to the specified requirements and intended behavior. The researchers created detailed test cases for each module, including user registration and authentication, online enrollment, course scheduling, intelligent instructor matching, student monitoring, chatbot assistance. Various testing scenarios were performed such as successful student registration, accurate enrollment in TDC and PDC courses, automatic instructor assignment based on availability, attendance monitoring. Manual testing was carried out by simulating actual user interactions under different system roles to ensure that all modules functioned cohesively and reliably.

During implementation, several minor issues were encountered. One issue involved scheduling conflicts caused by overlapping instructor availability during simultaneous bookings. This was resolved by refining the scheduling validation logic and implementing automatic conflict detection within the system. Another issue involved slight delays in chatbot response loading due to unstable local network connectivity. To optimize the initial page load and prioritize the critical



rendering path, the chatbot's heavy client-side dependencies were deferred using dynamic module imports, ensuring execution occurs only upon explicit user interaction

To ensure successful system adoption, orientation and training sessions were conducted for the administrators, staff, and driving instructors. The training focused on transitioning from manual processes such as physical scheduling boards, handwritten attendance monitoring, and manual record-keeping into a centralized digital platform. After completing the testing, debugging, and refinement stages, the implementation concluded successfully with a stable and functional web-based system capable of improving scheduling accuracy, reducing administrative workload, minimizing manual errors, and enhancing the operational efficiency of the BLRT Driving School.



Discussion of Findings

The results of the conducted testing demonstrate that the system performs effectively in terms of functionality, performance, security, usability, and user acceptance. Based on the findings, the system successfully achieved its intended objectives and provided reliable support for the operations of the BLRT Driving School management process.

In terms of **Functionality Testing (86% Success Rate)** the system successfully executed core features such as secure login authentication, malicious file upload prevention, student enrollment processing, intelligent instructor matching, and role-based access control. Most test cases passed successfully and produced the expected results. However, one issue was identified during the manual instructor assignment process where strict enrollment-specific instructor requirements prevented manual overrides. This finding highlights that while the automated matching mechanism effectively enforces system rules and maintains data consistency, additional administrative override procedures may still be required for exceptional cases.

For **Performance Testing (100% Success Rate)**, the system demonstrated fast response times and stable operation under normal workloads. Dashboard loading, document upload processing, intelligent instructor matching, and live search functionalities all executed within acceptable response thresholds. The intelligent instructor matching process completed within milliseconds, indicating that the system can efficiently process enrollment-related operations without causing delays to users. These results confirm that the system is capable of supporting daily operational activities in a responsive and efficient manner.

The **Security Testing (100% Success Rate)** results revealed that the system possesses strong protection against common web vulnerabilities and



unauthorized access attempts. All security test cases passed successfully, including brute force protection, SQL injection prevention, IDOR protection, cross-site scripting prevention, and malicious file upload blocking. The implementation of input sanitization, access restrictions, secure validation, and rate-limiting mechanisms contributed significantly to safeguarding user information and maintaining secure system operations. These findings indicate that the system provides a secure environment for both administrators and students.

Based on the **Usability Testing (4.73 Success Rate - Agree)** respondents provided highly positive feedback regarding the system's ease of use, interface design, and clarity of instructions. The system received high average scores across all usability criteria, particularly in user-friendliness and transaction feedback. Users found the system easy to navigate and understandable even with minimal effort, suggesting that the interface design effectively supports a smooth user experience. Although visual design and instructional clarity received slightly lower ratings compared to other criteria, they still remained within the positive evaluation range.

Similarly, the **Acceptance Testing (4.5 Success Rate - Agree)** results indicate a high level of overall satisfaction among end-users. Respondents agreed that the system functions as expected, performs efficiently, and meets operational requirements. The system's ability to provide understandable transaction feedback and maintain responsive performance contributed greatly to user acceptance. These findings demonstrate that the developed system is acceptable for practical implementation within the driving school environment.

Overall, the testing results highlight several strengths of the system, including reliable functionality, efficient performance, strong security measures,



and positive user experience. At the same time, the findings also suggest opportunities for further enhancement, particularly in improving administrative flexibility during instructor assignment procedures and refining interface presentation and instructional clarity. Future development may focus on implementing controlled manual override features, enhancing visual design elements, and further optimizing operational workflows to improve the overall effectiveness of the system.



Analysis of Results

I. Functional Testing

Functional testing was conducted to ensure that all modules and features of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT operate according to the specified requirements.

Test cases	Expected output	Actual output	Pass/Fail	Remarks
Login with correct credentials	User is redirected to dashboard	Work as expected	Pass	N/A
Upload a malicious .exe or unsupported .zip file.	The system rejects the file	Work as expected	Pass	N/A
Staff approves an enrollment but no instructor is available.	The system displays that no instructors are available.	Work as expected	Pass	N/A
Manually assign an instructor if the system finds no match.	The instructor is assigned to the student.	Manual assignment override failed due to non-compliance with enrollment-specific instructor requirements.	Fail	Implement manual override logic to bypass enrollment-specific instructor



					requirements.
Student enrollment submission	The system validates the input, and saves the record in the database.	Work as expected	Pass		N/A
Intelligent instructor matching	The system automatically identifies and assigns the available instructor.	Work as expected	Pass		N/A
Role-Based Access Control	Restricted document visibility is maintained for authorized personnel.	Work as expected	Pass		N/A

Table 3.24: Functional Testing.

Functional Testing Success Rate Description

The system successfully executed essential functions which included secure user login and malicious file upload protection and student enrollment processing and automated intelligent instructor matching and role-based access control maintenance. The system encountered a problem when it attempted to perform a manual instructor assignment because the automated matching process did not succeed. The manual override was blocked because the system



enforced strict compliance with instructor assignment rules which apply to specific enrollment scenarios therefore system developers need to create a special procedure that will allow staff members to bypass these regulations during manual assignment processes.

Calculations:

Success Rate = (Number of Passed Tests / Total Tests) $\times$ 100%

$$\text{Success Rate} = \frac{6}{7} \times 100 = 86\%$$

The system achieved an 86% success rate in Functional Testing. The single failure encountered was a minor formatting issue during the Manual assignment override failed due to non-compliance with enrollment-specific instructor requirements.



II. Performance Testing

Performance testing was executed to evaluate the system's responsiveness, processing speed, and overall stability under standard operational workloads typical for the BLRT Driving School environment.

Test Scenario	Expected Time	Actual Time(seconds)	Pass/Fail	Remarks
Loading the dashboard upon login	<3 sec	1.60 sec	Pass	N/A
Intelligent Instructor Matching	<3 sec	350 ms	Pass	N/A
Document upload processing	<5 sec	1.95 sec	Pass	N/A
Live search/filter users	<1sec	221 ms	Pass	N/A
Opening an uploaded document.	<3sec	905 ms	Pass	N/A

Table 3.25: Performance Testing



Performance Testing Success Rate Description

The system performed all standard operational tasks for the BLRT Driving School environment which included dashboard loading and intelligent instructor matching and document upload processing and live user searching and filtering operations at expected response times. The system achieved full success in all testing scenarios which demonstrated its ability to respond quickly without any performance issues. The intelligent instructor matching process completed its task in 350 milliseconds while live search took 221 milliseconds to execute which showed that the system can operate BLRT Driving School activities in an efficient manner.

Calculations:

Success Rate = (Number of Passed Tests / Total Tests) $\times$ 100%

$$\text{Success Rate} = \frac{5}{5} \times 100 = 100\%$$

The system achieved a perfect 100% success rate in Performance Testing, verifying that the system exceeded the expected time for every test scenario.



III. Security Testing

Security testing was conducted to ensure that the BLRT Driving School system adequately protects sensitive student information, uploaded documents, and core administrative modules against unauthorized access and common web vulnerabilities. This included verifying the system's defenses against brute-force attacks, SQL injection, cross-site scripting (XSS), and malicious file uploads, ensuring a secure environment for both students and staff.

Security Test	Expected Behavior	Actual Behavior	Pass/Fail	Remarks
Brute Force Protection	System throws an 'too many request' after 5 consecutive failed login	Work as expected	Pass	N/A
SQL injection(Login form)	System sanitizes inputs	Work as expected	Pass	N/A
Unauthorized admin access	System throws an 403(Forbidden) error	Work as expected	Pass	N/A
Insecure Direct Object	Student A attempting to view Student B's uploaded permit by changing the ID in the URL. The system should throw a 403(Forbidden) error	Work as expected	Pass	N/A
Cross-Site Scripting (XSS)	System escapes HTML tags in input fields (e.g.,	Work as expected	Pass	N/A



		<script>alert('XSS')</script>) to prevent script execution.			
Malicious upload	file	Attempting to upload a .php or .js file disguised as an image to the document module.	Work as expected	Pass	N/A

Table 3.26: Security Testing

Security Testing Success Rate Description

The system demonstrated strong defenses against common web vulnerabilities because it succeeded in all security test scenarios without any security breaches. The system protected itself against brute-force attacks by implementing a rate-limiting system which restricted users after they entered incorrect passwords five times. The system implemented input sanitization to block SQL injection attacks through login form validation which ensured that only authorized users could access admin functions. The system enforced strong access restrictions which stopped users from executing Insecure Direct Object Reference (IDOR) attacks to access other students' files and it successfully prevented Cross-Site Scripting (XSS) attacks through secure HTML tag escaping. The system effectively prevented users from uploading .php and .js files which they disguised as image files. The system demonstrated its ability to create a secure space that protected user data and operational activities.

Calculations:

Success Rate = (Number of Passed Tests / Total Tests) × 100%

$$Success Rate = \frac{6}{6} \times 100 = 100\%$$



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan
Email Address: bcc.sccp@yahoo.com



The system achieved a perfect 100% success rate in Security Testing, verifying that the application is highly secure, protected against basic database injection attacks, and strictly accessible only to legitimately verified and authorized personnel.



IV. Usability Testing

Usability testing was conducted among 21 selected respondents to systematically evaluate the interface design, ease of navigation, clarity of instructions, and overall user-friendliness of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT

Evaluation Criteria	Strongly Disagree (1)	Disagree (2)	Neutral (3)	Agree (4)	Strongly Agree (5)	Average Score
The system is easy to navigate	0	0	0	5	16	4.76
The interface design is clear and visually appealing	0	0	1	6	14	4.62
System instructions and labels are understandable	0	0	0	8	13	4.62



The system is user-friendly and requires minimal effort to learn	0	0	0	4	17	4.81
The system provides clear feedback for every transaction.	0	0	0	3	18	4.86

Table 3.27: Usability testing

Overall Average Usability Rating Description

The feedback gathered from the respondents indicates an overwhelmingly positive reception regarding the system's usability and design. Users rated the system's ability to deliver complete transaction feedback as the most important feature at 4.86 while they rated its user-friendliness at 4.81 because users find the application simple to learn. The interface design's visual appeal and system instructions received their lowest rating of 4.62 yet people still considered those elements to be above the positive "Agree" threshold while leaning toward "Strongly Agree" which demonstrates that the system delivers an outstanding user-centered experience through its comprehensive and understandable design.



Calculations:

Rating = Total Average Score in all Criteria / Total Number of Criteria.

$$\text{Rating} = \frac{4.76 + 4.62 + 4.62 + 4.81 + 4.86}{5} = \mathbf{4.73}$$

The overall average usability rating of the system is 4.73 (Strongly Agree). This confirms that the system user interface is exceptionally user-friendly, visually clear, easy to navigate, and effectively designed to support the daily operations of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT.



V. User Acceptance Testing

The User Acceptance Testing (UAT) analysis occurred with essential stakeholders and end users to confirm that the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT system meets its main operational goals while satisfying the needs of management, administrative staff, instructors, and students.

Evaluation Criteria	Strongly Disagree (1)	Disagree (2)	Neutral (3)	Agree (4)	Strongly Agree (5)	Average Score
System Functionality works as expected	0	0	1	5	15	4.67
System performance is fast and responsive	0	0	2	8	11	4.43



The system meets my needs and requirements	0	0	2	5	14	4.57
The interface design is clear and visually appealing.	0	0	2	9	10	4.38
System instructions and labels are understandable.	0	0	2	6	13	4.52

Table 3.28: User Acceptance Testing



Overall Average Acceptance Rating Description

The testing results show that all end-users of the system have achieved extremely high levels of acceptance and satisfaction with its performance. The system received its highest score for the assessment which measured its ability to deliver understandable transaction feedback at 4.86. The visual design of the interface and the system instructions achieved positive scores which reached 4.62 but these scores remained lower than their respective standards. The system receives positive feedback which shows it has simple navigation (4.76) and high user-friendliness (4.81) and it provides an easy-to-use experience for driving school management.

Calculations:

Rating = Total Average Score in all Criteria / Total Number of Criteria.

$$\text{Rating} = \frac{4.67 + 4.43 + 4.57 + 4.38 + 4.52}{5} = 4.5$$

The overall average acceptance rating of the system is 4.5 (Strongly Agree). This signifies that the system is widely accepted by its intended users, operationally sound, and fully capable of enhancing the efficiency, management, and daily operations of the Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT.



Chapter 4

RECOMMENDATIONS

Stakeholder

To fully optimize the digitized enrollment pipeline, it is highly recommended to integrate secure online payment gateways, such as GCash via unified application programming interfaces (APIs) like PayMongo. Incorporating real-time digital transaction processing will effectively bridge the gap between virtual document submission and enrollment confirmation, eliminating the necessity for students to visit the campus or business office physically for cash settlements. This expansion not only minimizes manual financial bookkeeping and administrative counter traffic for BLRT Driving School but also establishes a frictionless, end-to-end remote enrollment experience. Consequently, subsequent system iterations should prioritize embedding a robust, encrypted payment module to ensure transactional security, transparency, and data integrity.

Future Development

For future software developers seeking to extend the system's architecture, it is recommended to transition the platform's core code repositories into a containerized development environment using tools like Docker. Implementing containerization will ensure absolute environment consistency across different local workflows, effectively eliminating configuration disparities between development and production servers. Furthermore, future engineers should focus on refactoring the existing monolithic TALL stack structure to decouple the backend business logic into a RESTful API or GraphQL framework. This architectural separation will lay the necessary technical foundation for



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



seamlessly building and deploying future native mobile applications or integrating external third-party microservices.



References

- Atalaya, R., et al. (2023). BERT-Based Conversational AI for Streamlining University Enrollment in the Philippines. *Philippine Journal of Information Technology*.
- Bautista, L., & Fernandez, J. (2023). Dynamic Instructor Assignment Systems in Technical Schools. *Luzon ICT Journal*.
- Cortez, P., & Reyes, A. (2022). Web-Based Management Portals for Vocational Education. *Local IT Education Review*.
- Kumaran, S. (2024). Electronic Driving School Management: Replacing Manual Record-Keeping. *Journal of Modern Educational Tech*.
- Mayo, R. F. (2022). Web-Based Enrollment System for Public Junior High School in the Philippines: A Case Study in the Province of Cavite Public Schools. *Proceedings of the 2nd International Conference in Information and Computing Research (iCORE)*.
- Mendoza, K., & Cruz, V. (2024). Performance Analytics Integration in Region I Technical Schools. *Philippine Computing Journal*.
- Santos, R., & Lim, C. (2024). Automated Class and Teacher Schedulers in Philippine Schools. *Philippine Tech Deployment Review*.



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



APPENDICES

A. Endorsement Letter

 **Binalatongan Community College** 
Brgy. Ilang, San Carlos City, Pangasinan
Email Address: bcc.sccp@yahoo.com
Telephone Number: (075) 836-3384

December 3, 2025

*Received by:
Jimmy L. Padilla*

Mr. Jimmy L. Padilla
Administrator
BLRT Driving School
San Carlos City, Pangasinan

Subject: Request for Approval to Conduct Capstone Project Research in Your Organization

Dear Sir:

Greetings!

I am writing on behalf of our fourth-year Bachelor of Science in Information Technology (BSIT) students—**Joshua Q. Cabuang, Jim Spencer C. Lee Casay, Romark D. Pallsoc and Ryan M. Jacja**— who are required to undertake a research project as part of their Capstone Project. This serves as the Terminal Project Requirement for the BSIT program.

The Capstone Project aims to provide students with a deeper understanding of various organizational processes, transactions, and data environments. It enables them to design and develop computing solutions that address real-world challenges or needs within their chosen organization. As part of the project, the students are expected to deploy the developed computing solution (software/system/app) during the second semester of this school year.

In connection with this, we kindly request your approval to allow our students to conduct their study within your organization. Specifically, they propose to develop the **Web-Based Intelligent Driving School Management and Enrollment System with Automated Instructor Matching and Performance Analytics for BLRT** to address existing challenges and problems. Rest assured that all data and information collected for this study will be treated with the highest level of confidentiality and will be handled in full compliance with your organization's policies and the Data Privacy Act.

Should you need additional information regarding this matter, please feel free to contact me at **0909-555-4888** or via email at **brixon@binalatongan.edu.ph**.

We greatly appreciate your support and cooperation in helping our students fulfill this important academic requirement.

Thank you very much, and we look forward to your favorable response.

Sincerely,


BRIXON J. DE VERA, MIT
Dean / Capstone Project Adviser

Approved by:
Name of Organization: _____
Name of Representative: _____
Position: _____
Signature: _____
Date: _____



B. System Source Codes

B.1 Login

AttemptToAuthenticate.php(Core authentication attempt)

```
<?php
namespace Laravel\Fortify\Actions;
use Illuminate\Auth\Events\Failed;
use Illuminate\Contracts\Auth\StatefulGuard;
use Illuminate\Validation\ValidationException;
use Laravel\Fortify\Fortify;
use Laravel\Fortify>LoginRateLimiter;
class AttemptToAuthenticate
{
    /**
     * The guard implementation.
     *
     * @var \Illuminate\Contracts\Auth\StatefulGuard
     */
    protected $guard;

    /**
     * The login rate limiter instance.
     *
     * @var \Laravel\Fortify>LoginRateLimiter
     */
    protected $limiter;

    /**
     * Create a new controller instance.
     *
     * @param \Illuminate\Contracts\Auth\StatefulGuard $guard
```



```
* @param \Laravel\Fortify>LoginRateLimiter $limiter
* @return void
*/
public function __construct(StatefulGuard $guard, LoginRateLimiter $limiter)
{
    $this->guard = $guard;
    $this->limiter = $limiter;
}
/**
 * Handle the incoming request.
 *
 * @param \Illuminate\Http\Request $request
 * @param callable $next
 * @return mixed
 */
public function handle($request, $next)
{
    if (Fortify::$authenticateUsingCallback) {
        return $this->handleUsingCustomCallback($request, $next);
    }
    if ($this->guard->attempt(
        $request->only(Fortify::username(), 'password'),
        $request->boolean('remember'))
    ) {
        return $next($request);
    }
    $this->throwFailedAuthenticationException($request);
}
```



/**

* Attempt to authenticate using a custom callback.

*

* @param \Illuminate\Http\Request \$request

* @param callable \$next

* @return mixed

*/

protected function handleUsingCustomCallback(\$request, \$next)

{

 \$user = call_user_func(Fortify::\$authenticateUsingCallback, \$request);

 if (! \$user) {

 \$this->fireFailedEvent(\$request);

 return \$this->throwFailedAuthenticationException(\$request);

 }

 \$this->guard->login(\$user, \$request->boolean('remember'));

 return \$next(\$request);

}

/**

* Throw a failed authentication validation exception.

*

* @param \Illuminate\Http\Request \$request

* @return void

*

* @throws \Illuminate\Validation\ValidationException

*/

protected function throwFailedAuthenticationException(\$request)

{

 \$this->limiter->increment(\$request);



```
        throw ValidationException::withMessages([
            Fortify::username() => [trans('auth.failed')],
        ]);
    }
}

/**
 * Fire the failed authentication attempt event with the given arguments.
 *
 * @param \Illuminate\Http\Request $request
 * @return void
 */
protected function fireFailedEvent($request)
{
    event(new Failed($this->guard->name ?? config('fortify.guard'), null, [
        Fortify::username() => $request->{Fortify::username()},
        'password' => $request->password,
    ]));
}
}
```




B.2 Registration

RegisteredUserController.php

```
<?php
namespace Laravel\Fortify\Http\Controllers;
use Illuminate\Auth\Events\Registered;
use Illuminate\Contracts\Auth\StatefulGuard;
use Illuminate\Http\Request;
use Illuminate\Routing\Controller;
use Illuminate\Support\Str;
use Laravel\Fortify\Contracts\CreatesNewUsers;
use Laravel\Fortify\Contracts\RegisterResponse;
use Laravel\Fortify\Contracts\RegisterViewResponse;
use Laravel\Fortify\Fortify;
class RegisteredUserController extends Controller
{
    /**
     * The guard implementation.
     *
     * @var \Illuminate\Contracts\Auth\StatefulGuard
     */
    protected $guard;

    /**
     * Create a new controller instance.
     *
     * @param \Illuminate\Contracts\Auth\StatefulGuard $guard
     * @return void
     */
    public function __construct(StatefulGuard $guard)
```



```
{
    $this->guard = $guard;
}
/**
 * Show the registration view.
 *
 * @param \Illuminate\Http\Request $request
 * @return \Laravel\Fortify\Contracts\RegisterViewResponse
 */
public function create(Request $request): RegisterViewResponse
{
    return app(RegisterViewResponse::class);
}
/**
 * Create a new registered user.
 *
 * @param \Illuminate\Http\Request $request
 * @param \Laravel\Fortify\Contracts\CreatesNewUsers $creator
 * @return \Laravel\Fortify\Contracts\RegisterResponse
 */
public function store(Request $request,
                      CreatesNewUsers $creator): RegisterResponse
{
    if (config('fortify.lowercase_usernames') &&
$request->has(Fortify::username())) {
        $request->merge([
            Fortify::username() => Str::lower($request->{Fortify::username()}),
        ]);
    }
}
```



```
}  
event(new Registered($user = $creator->create($request->all())));  
$this->guard->login($user, $request->boolean('remember'));  
if ($request->hasSession()) {  
    $request->session()->regenerate();  
}  
return app(RegisterResponse::class);  
}  
}
```

FortifyServiceProvider.php(Customization of authentication)

```
<?php  
namespace App\Providers;  
use App\Actions\Fortify\CreateNewUser;  
use App\Actions\Fortify\ResetUserPassword;  
use Illuminate\Cache\RateLimiting\Limit;  
use Illuminate\Http\Request;  
use Illuminate\Support\Facades\RateLimiter;  
use Illuminate\Support\ServiceProvider;  
use Illuminate\Support\Str;  
use Laravel\Fortify\Fortify;  
use Illuminate\Support\Facades\Auth;  
use Laravel\Fortify\Contracts\RegisterResponse;  
class FortifyServiceProvider extends ServiceProvider  
{  
    /**  
     * Register any application services.  
     */  
    public function register(): void
```



```
{
    //
}
/**
 * Bootstrap any application services.
 */
public function boot(): void
{
    $this->configureActions();
    $this->configureViews();
    $this->configureRateLimiting();
    $this->app->singleton(RegisterResponse::class, function () {
        return new class implements RegisterResponse {
            public function toResponse($request)
            {
                $user = Auth::user();
                // Check role and redirect
                if ($user->hasRole('Student')) {
                    return redirect()->route('student_profile.create');
                }
                // Check for role Instructor
                if ($user->hasRole('Instructor')) {
                    return redirect()->route('instructor_profile.create');
                }
                // Check for role Staff
                if ($user->hasRole('Staff')) {
                    return redirect('/dashboard');
                }
            }
        };
    });
}
```



```
// Default location for everyone else
return redirect('/dashboard');
    }
};
});
}
/**
 * Configure Fortify actions.
 */
private function configureActions(): void
{
    Fortify::resetUserPasswordsUsing(ResetUserPassword::class);
    Fortify::createUsersUsing(CreateNewUser::class);
}
/**
 * Configure Fortify views.
 */
private function configureViews(): void
{
    Fortify::loginView(fn () => view('pages::auth.login'));
    Fortify::verifyEmailView(fn () => view('pages::auth.verify-email'));
    Fortify::twoFactorChallengeView(fn () =>
view('pages::auth.two-factor-challenge'));
    Fortify::confirmPasswordView(fn () =>
view('pages::auth.confirm-password'));
    Fortify::registerView(fn () => view('pages::auth.register'));
    Fortify::resetPasswordView(fn () => view('pages::auth.reset-password'));
```



```

Fortify::requestPasswordResetLinkView(fn    ()    =>
view('pages::auth.forgot-password'));
}
/**
 * Configure rate limiting.
 */
private function configureRateLimiting(): void
{
    RateLimiter::for('two-factor', function (Request $request) {
        return Limit::perMinute(5)->by($request->session()->get('login.id'));
    });
    RateLimiter::for('login', function (Request $request) {
                                                                    $throttleKey    =
Str::transliterate(Str::lower($request->input(Fortify::username()))).'|'.$request->i
p());
        return Limit::perMinute(5)->by($throttleKey);
    });
}
}

```



B.3 Routes

web.php

<?php

```
use Illuminate\Foundation\Auth\EmailVerificationRequest;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Route;
```

```
use Illuminate\Support\Facades\Storage;
```

```
// PWA offline fallback — must remain outside all middleware groups to prevent
```

```
// redirect loops for both authenticated and unauthenticated users when offline.
```

```
Route::view('/offline', 'offline')->name('offline');
```

```
Route::get('/email/verify', function () {
```

```
    return view('pages.auth.verify-email');
```

```
})->middleware('auth')->name('verification.notice');
```

```
// The Link Handler (When they click the email button)
```

```
Route::get('/email/verify/{id}/{hash}', function (EmailVerificationRequest  
$request) {
```

```
    $request->fulfill();
```

```
    return redirect('/dashboard');
```

```
})->middleware(['auth', 'signed'])->name('verification.verify');
```

```
// Resend Verification Email
```

```
Route::post('/email/verification-notification', function (Request $request) {
```

```
    $request->user()->sendEmailVerificationNotification();
```

```
    return back()->with('status', 'verification-link-sent');
```

```
})->middleware(['auth', 'throttle:6,1'])->name('verification.send');
```

```
Route::middleware(['guest'])->group(function () {
```

```
    Route::view('/', 'welcome')->name('home');
```

```
    Route::view('/services', 'services')->name('guest.services');
```

```
    Route::view('/about-us', 'about-us')->name('guest.about');
```



```
});  
Route::middleware(['auth', 'verified'])->group(function () {  
    Route::livewire('/student/onboard', 'pages::student.onboard')  
        ->name('student_profile.create');  
    Route::livewire('instructor/onboard', 'pages::instructor.onboard')  
        ->name('instructor_profile.create');  
    // Pages allowed only when the users already created their profiles  
    Route::middleware(['verified', 'profile_completed'])->group(function () {  
        Route::view('dashboard', 'pages::dashboard')->name('dashboard');  
        // Student documents  
        Route::livewire('student/upload-document',  
'pages::student.upload-document')  
            ->name('document.upload');  
        // Enrollment form  
        Route::livewire('student/enrollment/{course}',  
'pages::student.enrollment-form')  
            ->name('enrollment.create');  
        Route::livewire('student/my-schedule', 'pages::student.my-schedule')  
            ->name('student.my-schedule');  
        Route::livewire('student/academic-records',  
'pages::student.academic-records')  
            ->name('student.academic-records');  
        Route::livewire('student/performance-analytics/{enrollment}',  
'pages::student.performance-analytics')  
            ->name('student.performance-analytics');  
    });  
    // Admin routes  
    Route::middleware(['can:user.view'])->group(function () {
```




```
//Registration management
Route::livewire('pending-registrations',
'pages::admin.pending-registrations')
->name('admin.pending-registrations');
Route::livewire('admin/registrations/{instructor}',
'pages::admin.registration-data')
->name('admin.registration-data');
// Course management
Route::livewire('manage-courses', 'pages::admin.manage-course')
->name('admin.manage-courses');
// Document management
Route::livewire('pending-documents', 'pages::admin.pending-documents')
->name('admin.pending-documents');
// Vehicle management
Route::livewire('manage-vehicle', 'pages::admin.manage-vehicle')
->name('admin.manage-vehicle');
// Serve private documents
Route::get('document/serve/{document}', function (App\Models\Document
$document) {
    return Storage::disk('local')->response($document->file_path);
})->name('admin.document.serve');
// Document checking
Route::livewire('document/{document}', 'pages::admin.document')
->name('admin.document.check');
// Instructor Performance Analytics
Route::livewire('instructor-performances',
'pages::admin.instructor-performances')
->name('admin.instructor-performances');
```



```
Route::livewire('instructor/{instructor}/evaluations',
'pages::admin.instructor-evaluations')
    ->name('admin.instructor.evaluations');
// User management
Route::livewire('manage-users', 'pages::admin.manage-users')
    ->name('admin.manage-users');
});
Route::middleware(['can:enrollment.view_any'])->group(function () {
    Route::get('document/serve/{document}', function (App\Models\Document
$document) {
        return Storage::disk('local')->response($document->file_path);
    })->name('admin.document.serve');
// Document checking
Route::livewire('document/{document}', 'pages::admin.document')
    ->name('admin.document.check');
});
// Staff routes
Route::middleware(['profile_completed',
'can:student.view_any'])->group(function () {
// Enrollment management
Route::livewire('manage-enrollments', 'pages::staff.enrollments')
    ->name('staff.manage-enrollments');
Route::livewire('enrollment/{enrollment}', 'pages::staff.enrollment')
    ->name('staff.enrollment.show');
Route::livewire('approved-enrollments',
'pages::staff.approved-enrollments')
    ->name('staff.approved-enrollments');
```



```
Route::livewire('approved-enrollment/{enrollment}',
'pages::staff.approved-enrollment')
    ->name('staff.approved-enrollment.show');
Route::livewire('waiting-list', 'pages::staff.waiting-list')
    ->name('staff.waiting-list');
Route::livewire('accredited-clinics', 'pages::admin.accredited-clinics')
    ->name('admin.accredited-clinics');
});
// Instructor routes
Route::middleware(['can:instructor.view_own']->group(function () {
    Route::livewire('my-schedule', 'pages::instructor.my-schedule')
        ->name('instructor.my-schedule');
    Route::livewire('my-students', 'pages::instructor.my-students')
        ->name('instructor.my-students');
        Route::livewire('my-students/{enrollment}',
'pages::instructor.view-student')
            ->name('instructor.student.show');
            Route::livewire('assessment/{enrollment}/{bookingSession}',
'pages::instructor.assessment')
                ->name('instructor.assessment');
                Route::livewire('performance-reviews',
'pages::instructor.performance-reviews')
                    ->name('instructor.performance-reviews');
                });
            });
require __DIR__ . '/settings.php';
```



B.4 Dashboards

admin-dashboard.blade.php

```
<?php
use Livewire\Component;
use App\Models\Document;
use App\Models\Enrollment;
use App\Models\EnrollmentForm;
use App\Models\InstructorProfile;
use App\Services\InstructorPerformanceService;
use Livewire\Attributes\Computed;
use Carbon\Carbon;

new class extends Component {
    public $searchInstructor = "";
    #[Computed]
    public function pendingDocsCount()
    {
        return Document::where('status', 'pending')->count();
    }
    #[Computed]
    public function revenueData()
    {
        $now = Carbon::now();
        $thisMonth = Enrollment::whereMonth('created_at',
$now->month)->whereYear('created_at', $now->year)->sum('amount_paid');
        $lastMonth = Enrollment::whereMonth('created_at',
$now->copy()->subMonth()->month)
->whereYear('created_at', $now->copy()->subMonth()->year)
->sum('amount_paid');
```



```
$difference = $thisMonth - $lastMonth;
$trend = $lastMonth > 0 ? ($difference / $lastMonth) * 100 : 0;
return [
    'value' => $thisMonth,
    'trend' => number_format($trend, 1) . '%',
    'trend_color' => $trend >= 0 ? 'emerald' : 'rose',
    'subtext' => 'vs last month: ₱' . number_format($lastMonth, 2),
];
}
#[Computed]
public function enrollmentStats()
{
    $active = Enrollment::where('status', 'active')->count();
    $tdc = Enrollment::where('status', 'active')
        ->whereHas('course', function ($q) {
            $q->where('type', 'theoretical');
        })
        ->count();
    $pdc = Enrollment::where('status', 'active')
        ->whereHas('course', function ($q) {
            $q->whereIn('type', ['practical', 'comprehensive']);
        })
        ->count();
    return [
        'total' => $active,
        'tdc' => $tdc,
        'pdc' => $pdc,
    ];
}
```



```
}  
#[Computed]  
public function pendingActions()  
{  
    $forms = EnrollmentForm::where('status', 'submitted')->count();  
    $docs = $this->pendingDocsCount;  
    return [  
        'total' => $forms + $docs,  
        'forms' => $forms,  
        'docs' => $docs,  
    ];  
}  
#[Computed]  
public function passedStudentsCount()  
{  
    $tdc = Enrollment::where('final_result', 'pass')  
        ->whereHas('course', function ($query) {  
            $query->where('type', 'theoretical');  
        })  
        ->count();  
    $pdc = Enrollment::where('final_result', 'pass')  
        ->whereHas('course', function ($query) {  
            $query->whereIn('type', ['practical', 'comprehensive']);  
        })  
        ->count();  
    $totalTdc = Enrollment::whereHas('course', function ($query) {  
        $query->where('type', 'theoretical');  
    })->count();  
}
```



```
$totalPdc = Enrollment::whereHas('course', function ($query) {
    $query->whereIn('type', ['practical', 'comprehensive']);
})->count();
return [
    'tdc' => $tdc,
    'pdc' => $pdc,
    'total_tdc' => $totalTdc,
    'total_pdc' => $totalPdc,
];
}
#[Computed]
public function instructorsPerformances()
{
    $service = app(InstructorPerformanceService::class);
    $query = InstructorProfile::with('user')->where('status',
'verified')->where('is_active', true);
    if (!empty($this->searchInstructor)) {
        return $query->whereHas('user', function($q) {
            $q->where('name', 'like', '%' . $this->searchInstructor . '%');
        })->take(4)->get();
    }
    $allInstructors = $query->get();
    $preview = collect();
    // 1. Find the best TDC instructor (theoretical)
    $tdcInstructor = $allInstructors->first(function ($instructor) {
        return $instructor->enrollments()->whereHas('course', fn($q) =>
$q->where('type', 'theoretical'))->exists();
    });
});
```



```
if ($tdcInstructor) {
    $preview->push($tdcInstructor);
}

// 2. Find a different PDC instructor (practical/comprehensive)
$pdciInstructor = $allInstructors->where('id', '!=',
$tdcInstructor?->id)->first(function ($instructor) {
    return $instructor->enrollments()->whereHas('course', fn($q) =>
$q->whereIn('type', ['practical', 'comprehensive']))->exists();
});
if ($pdciInstructor) {
    $preview->push($pdciInstructor);
}
return $preview;
}

#[Computed]
public function pendingDocuments()
{
    return Document::with('user')
        ->where('status', 'pending')
        ->latest()
        ->take(5)
        ->get();
}

};

?>
```

student-dashboard.php

<?php



```
use Livewire\Component;
use App\Models\Document;
use App\Models\Course;
use Livewire\Attributes\Computed;
use Illuminate\Support\Facades\Auth;
new class extends Component {
    protected array $blockingEnrollmentStatuses = ['active', 'pending',
'waiting_list'];
    // Check if the student uploaded at least one document
    #[Computed]
    public function hasDocument()
    {
        return Document::where('user_id', Auth::user()->id)->exists();
    }
    #[Computed]
    public function requiredDocumentTypes()
    {
        $profile = Auth::user()->studentProfile;

        $types = ['medical', 'adl_form', 'valid_id'];

        if ($profile) {
            if ($profile->nationality === 'foreigner') {
                $types[] = 'passport';
            } else {
                $types[] = 'birth_cert';
            }
        }
    }
}
```



```
$types[] = 'birth_cert'; // Default
}

return $types;
}
#[Computed]
public function uploadedDocumentsCount()
{
    $requiredTypes = $this->requiredDocumentTypes;

    return Document::where('user_id', Auth::user()->id)
        ->whereIn('type', $requiredTypes)
        ->distinct('type')
        ->count('type');
}
#[Computed]
public function isComplete()
{
    return Document::where('user_id', Auth::user()->id)
        ->where('status', 'verified')
        ->exists();
}
#[Computed]
public function courses()
{
    return Course::query()->select('id', 'title', 'description', 'price',
'type')->get();
}
```



```
//Get the active current enrollment for the student
#[Computed]
public function currentEnrollment()
{
    return Auth::user()
        ->studentProfile
        ->enrollments()
        ->where('status', 'active')
        ->first();
}
#[Computed]
public function hasBlockingEnrollment()
{
    $studentProfile = Auth::user()->studentProfile;
    if (!$studentProfile) {
        return false;
    }
    return $studentProfile->enrollments()
        ->whereIn('status', $this->blockingEnrollmentStatuses)
        ->exists();
}
#[Computed]
public function blockingEnrollmentStatus()
{
    $studentProfile = Auth::user()->studentProfile;
    if (!$studentProfile) {
        return null;
    }
}
```



```
$statuses = $studentProfile->enrollments()
    ->whereIn('status', $this->blockingEnrollmentStatuses)
    ->pluck('status');
foreach (['pending', 'active', 'waiting_list'] as $priorityStatus) {
    if ($statuses->contains($priorityStatus)) {
        return $priorityStatus;
    }
}
return null;
}

#[Computed]
public function hasPendingEnrollmentForm()
{
    $studentProfile = Auth::user()->studentProfile;
    if (!$studentProfile) {
        return false;
    }
    return $studentProfile->enrollmentForms()
        ->where('status', 'submitted')
        ->exists();
}

#[Computed]
public function isEnrollmentBlocked()
{
    return $this->hasBlockingEnrollment || $this->hasPendingEnrollmentForm;
}

#[Computed]
public function enrollmentBlockReason()
```



```
{
    if ($this->blockingEnrollmentStatus === 'pending') {
        return 'Your enrollment is currently pending.';
    }
    if ($this->blockingEnrollmentStatus === 'active') {
        return 'You already have an active enrollment.';
    }
    if ($this->blockingEnrollmentStatus === 'waiting_list') {
        return 'You are currently on the waiting list for enrollment.';
    }
    if ($this->hasPendingEnrollmentForm) {
        return 'Your submitted enrollment form is still under review.';
    }
    return null;
}

#[Computed]
public function hasCompletedTdc()
{
    $studentProfile = Auth::user()->studentProfile;
    if (!$studentProfile) {
        return false;
    }

    // Student is considered done with TDC if they have an enrollment with
    tdc_status = 'completed'
    // or a theoretical course enrollment that is marked completed/active.
    // We will rely on tdc_status = 'completed'.
    return $studentProfile->enrollments()
        ->where('tdc_status', 'completed')
```



```
->exists());  
}  
#[Computed]  
public function progressData()  
{  
    $enrollment = $this->currentEnrollment;  
    if (!$enrollment) {  
        return [  
            'percent' => 0,  
            'milestone' => 'No active enrollment',  
        ];  
    }  
    $course = $enrollment->course;  
    $isTdc = $course->type === 'theoretical';  
    $percent = (int) $enrollment->progress_percent;  
    if ($isTdc) {  
        $completed = (float) $enrollment->tdc_hours_completed;  
        $required = (float) $enrollment->tdc_hours_required;  
        $milestone = $completed >= $required  
    } else {  
        $completed = (float) $enrollment->pdc_hours_completed;  
        $required = (float) $enrollment->pdc_hours_required;  
        $milestone = $completed >= $required  
            ? 'Next Milestone: Practical Exam'  
            : "Driving Hours: {$completed} / {$required}";  
    }  
    return [  
        'percent' => $percent,
```



```
'milestone' => $milestone,  
    ];  
}  
};  
?>
```

instructor-dashboard.blade.php

```
<?php  
use Livewire\Component;  
use App\Models\InstructorProfile;  
use App\Models\BookingSession;  
use App\Models\User;  
use Illuminate\Support\Facades\Auth;  
use App\Models\InstructorMetric;  
use Livewire\Attributes\Computed;  
new class extends Component {  
    public $accepting_sessions;  
    public function mount()  
    {  
        $profile = Auth::user()->instructorProfile;  
        if ($profile) {  
            $this->accepting_sessions = $profile->status === 'verified';  
        }  
    }  
    public function updatedAcceptingSessions($value)  
    {  
        $profile = Auth::user()->instructorProfile;  
        if ($profile && in_array($profile->status, ['verified', 'not_accepting',  
'on_leave'])) {
```



```
$profile->update([
    'status' => $value ? 'verified' : 'not_accepting',
]);
}
}
#[Computed]
public function metrics()
{
    $profile = Auth::user()->instructorProfile;
    if (!$profile) {
        return (object) []; // or sensible default
    }
    $instructorId = $profile->id;
    $metric = InstructorMetric::where('instructor_id', $instructorId)
        ->where('metric_month', now()->startOfMonth()->format('Y-m-d'))
        ->first();
    if (!$metric) {
        return (object) [
            'metric_month' => now()->startOfMonth(),
            'total_sessions' => 0,
            'completed_sessions' => 0,
            'total_hours' => 0,
            'avg_rating' => 0,
            'students_taught' => 0,
            'students_passed' => 0,
            'pass_rate' => 0,
        ];
    }
}
```




```
        return $metric;
    }
    #[Computed]
    public function courses()
    {
        $profile = Auth::user()->instructorProfile;
        if (!$profile) {
            return collect();
        }

        return \App\Models\Course::whereHas('enrollments', function($q) use
($profile) {
            $q->where('instructor_id', $profile->id)
                ->whereHas('instructorPerformances');
        }->take(2)->get();
    }
    #[Computed]
    public function todaySessions()
    {
        $profile = Auth::user()->instructorProfile;
        if (!$profile) return collect();
        return BookingSession::where('instructor_id', $profile->id)
            ->whereDate('start_time', today())
            ->with(['enrollment.studentProfile.user', 'enrollment.course'])
            ->orderBy('start_time')
            ->get();
    }
    #[Computed]
    public function licenseInfo()
```



```
{
    $profile = Auth::user()->instructorProfile;
    if (!$profile) return null;
    $expiry = $profile->license_expiry;
    $now = now();
    $isExpired = $expiry ? $expiry->isPast() : false;
    $daysRemaining = $expiry ? (int) $now->diffInDays($expiry, false) : null;
    return (object) [
        'number' => $profile->license_number,
        'expiry' => $expiry,
        'is_expired' => $isExpired,
        'days_remaining' => $daysRemaining,
        'is_expiring_soon' => $daysRemaining !== null && $daysRemaining <=
90 && $daysRemaining > 0,
    ];
}
};
?>
```



staff-dashboard.blade.php

```
<?php
use Livewire\Component;
use App\Models\EnrollmentForm;
use Livewire\Attributes\Computed;
new class extends Component {
    #[Computed]
    public function enrollmentStatus()
    {
        return [
            'pending' => EnrollmentForm::where('status', 'submitted')->count(),
            'approved' => EnrollmentForm::where('status', 'approved')->count(),
            'rejected' => EnrollmentForm::where('status', 'rejected')->count(),
        ];
    }
};
?>
<div class="flex h-full w-full flex-1 flex-col gap-6 rounded-xl font-sans
text-slate-900 dark:text-slate-100">
    @if(auth()->user()->status === 'active')
        {{-- HEADER: Operational Overview --}}
        <div>
            <flux:heading size="xl" class="text-2xl font-bold tracking-tight">Staff
Operations Dashboard</flux:heading>
            <flux:text>
                {{ now()->format('l, F j, Y') }} • Daily Operational Oversight
            </flux:text>
        </div>
    @endif
</div>
```



```

</div>
<div class="w-full">
  <livewire:staff.master-schedule />
</div>
@elseif(auth()->user()->status === 'pending')
  <div class="flex flex-col items-center justify-center h-full py-12 px-6
text-center bg-amber-50/50 dark:bg-amber-900/10 border border-amber-200
dark:border-amber-800/50 rounded-2xl">
    <div class="p-4 bg-amber-100 dark:bg-amber-900/30 text-amber-600
dark:text-amber-400 rounded-full mb-6">
      <flux:icon icon="clock" class="size-12" />
    </div>
    <flux:heading size="xl" class="mb-2">Account Pending
Verification</flux:heading>
    <flux:text class="max-w-md mx-auto text-slate-600
dark:text-slate-400">
      Your staff account is currently under review. Please wait until an
administrator verifies your account before you can access staff operations.
    </flux:text>
  </div>
@elseif(auth()->user()->status === 'rejected')
  <div class="flex flex-col items-center justify-center h-full py-12 px-6
text-center bg-red-50/50 dark:bg-red-900/10 border border-red-200
dark:border-red-800/50 rounded-2xl">
    <div class="p-4 bg-red-100 dark:bg-red-900/30 text-red-600
dark:text-red-400 rounded-full mb-6">
      <flux:icon icon="x-circle" class="size-12" />
    </div>

```



<flux:heading size="xl" class="mb-2">Account Not
Approved</flux:heading>

<flux:text class="max-w-md mx-auto text-slate-600
dark:text-slate-400">

Unfortunately, your staff application has been rejected. Please contact
an administrator for more information.

</flux:text>

</div>

@endif

</div>



C. Database Schema

Name	users	Primary	id	Search column name...						
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ auto_increment	EMPTY	→ EMPTY	
2	name	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
3	email	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
4	email_verified_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
5	password	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
6	status	enum('active','pending','rejected')	✓ utf8mb4	utf8mb4_unicode_ci	NO	active	✓ EMPTY	EMPTY	→ EMPTY	
7	two_factor_secret	text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
8	two_factor_recovery_codes	text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
9	two_factor_confirmed_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
10	remember_token	varchar(100)	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	

Figure C.1 Users

Name	student_profiles	Primary	id							
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ auto_increment	EMPTY	→ EMPTY	
2	user_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	users(id)	→ EMPTY	
3	birth_date	date	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
4	contact_number	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
5	address	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
6	nationality	enum('filipino','foreigner')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
7	is_minor	tinyint(1)	✓ NULL	NULL	NO	0	✓ EMPTY	EMPTY	→ EMPTY	
8	occupation	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
9	educational_attainment	enum('elementary','high_school','college','post_graduate')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
10	civil_status	enum('single','married','widowed','separated')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
11	sex	enum('male','female')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
12	ltms_client_id	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
13	student_permit_or_license_no	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
14	is_active	tinyint(1)	✓ NULL	NULL	NO	0	✓ EMPTY	EMPTY	→ EMPTY	
15	meta_details	json	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
16	created_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
17	updated_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	

Figure C.2 Student_Profiles



Name		Primary								
instructor_profiles		id								
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	→ EMPTY	
2	user_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	users(id)	→ EMPTY	
3	license_number	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY	
4	license_expiry	date	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
5	skills	json	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
6	vehicle_types	json	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
7	weekly_schedule	json	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
8	is_active	tinyint(1)	NULL	NULL	NO	0	EMPTY	EMPTY	→ EMPTY	
9	status	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	pending	EMPTY	EMPTY	→ EMPTY	
10	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	
11	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	

Figure C.3 Instructor_Profiles

Name		Primary								
instructor_metrics		id								
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	→ EMPTY	
2	instructor_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	instructor_profiles(id)	→ EMPTY	
3	metric_month	date	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
4	total_sessions	int	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
5	completed_sessions	int	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
6	total_hours	decimal(10,2)	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
7	avg_rating	decimal(3,2)	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
8	students_taught	int	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
9	students_passed	int	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
10	pass_rate	decimal(5,2)	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
11	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	
12	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	

Figure C.4 Instructor_Metrics



Name assessments		Primary	id									Search column name...	
#	column_name		data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment			
1	id		bigint unsigned	✓ NULL	NUL	NO	NUL	✓ auto_increment	EMPTY	→ EMPTY			
2	enrollment_id		bigint unsigned	✓ NULL	NUL	NO	NUL	✓ EMPTY	enrollments(id)	→ EMPTY			
3	student_id		bigint unsigned	✓ NULL	NUL	NO	NUL	✓ EMPTY	student_profiles(id)	→ EMPTY			
4	instructor_id		bigint unsigned	✓ NULL	NUL	NO	NUL	✓ EMPTY	instructor_profiles(id)	→ EMPTY			
5	booking_session_id		bigint unsigned	✓ NULL	NUL	YES	NUL	✓ EMPTY	booking_sessions(id)	→ EMPTY			
6	assessment_type		enum('formative','summative','practical','theoretical')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NUL	✓ EMPTY	EMPTY	→ EMPTY			
7	assessment_date		date	✓ NULL	NUL	NO	NUL	✓ EMPTY	EMPTY	→ EMPTY			
8	pre_drive_checklist		json	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
9	pre_drive_errors		int	✓ NULL	NUL	NO	0	✓ EMPTY	EMPTY	→ EMPTY			
10	immediate_fails		json	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
11	immediate_fail_count		int	✓ NULL	NUL	NO	0	✓ EMPTY	EMPTY	→ EMPTY			
12	driving_skills		json	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
13	driving_skills_rating		int	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ 1-Poor, 2-Fair, 3-Good			
14	number_of_attempts		int	✓ NULL	NUL	NO	1	✓ EMPTY	EMPTY	→ EMPTY			
15	traffic_rules		json	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
16	traffic_rules_rating		int	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ 1-Poor, 2-Fair, 3-Good			
17	learner_type		enum('slow','fast','hesitant')	✓ utf8mb4	utf8mb4_unicode_ci	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
18	recommended_for_additional_driving		tinyint(1)	✓ NULL	NUL	NO	0	✓ EMPTY	EMPTY	→ EMPTY			
19	instructor_remarks		text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
20	observations		text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
21	is_passed		tinyint(1)	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
22	failure_reason		text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
23	created_at		timestamp	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			
24	updated_at		timestamp	✓ NULL	NUL	YES	NUL	✓ EMPTY	EMPTY	→ EMPTY			

Figure C.5 Assessments

Name	booking_sessions	Primary	id						
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ auto_increment	EMPTY	→ EMPTY
2	enrollment_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	enrollments(id)	→ EMPTY
3	instructor_id	bigint unsigned	✓ NULL	NULL	YES	NULL	✓ EMPTY	instructor_profiles(id)	→ EMPTY
4	vehicle_id	bigint unsigned	✓ NULL	NULL	YES	NULL	✓ EMPTY	vehicles(id)	→ EMPTY
5	start_time	datetime	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY
6	end_time	datetime	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
7	type	enum('lecture','driving','assessment')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY
8	status	enum('scheduled','completed','cancelled','no-show')	✓ utf8mb4	utf8mb4_unicode_ci	NO	scheduled	✓ EMPTY	EMPTY	→ EMPTY
9	score	decimal(5,2)	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
10	is_passed	tinyint(1)	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
11	rating	int	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
12	review_comment	text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
13	skill_progress	json	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
14	change_log	json	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
15	created_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY
16	updated_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY

Figure C.6 Booking_Sessions



Name		Primary		id						
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ auto_increment	EMPTY	→ EMPTY	
2	code	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
3	title	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
4	description	text	✓ utf8mb4	utf8mb4_unicode_ci	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
5	price	decimal(8,2)	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
6	duration_hours	int	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
7	type	enum('theoretical','practical')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
8	prerequisites	json	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
9	is_active	tinyint(1)	✓ NULL	NULL	NO	1	✓ EMPTY	EMPTY	→ EMPTY	
10	created_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
11	updated_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	

Figure C.7 Courses

Name		Primary		id						
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ auto_increment	EMPTY	→ EMPTY	
2	user_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	users(id)	→ EMPTY	
3	type	enum('birth_cert','medical','adl_form','valid_id','marriage_contract','tdc_	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
4	file_path	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
5	status	enum('pending','verified','rejected')	✓ utf8mb4	utf8mb4_unicode_ci	NO	pending	✓ EMPTY	EMPTY	→ EMPTY	
6	metadata	json	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
7	verified_by	bigint unsigned	✓ NULL	NULL	YES	NULL	✓ EMPTY	users(id)	→ EMPTY	
8	created_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
9	updated_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	

Figure C.8 Documents



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	NO	NULL	auto_increment		
2	student_id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	NO	NULL		student_profiles(id)	
3	course_id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	NO	NULL		courses(id)	
4	control_number	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL			Top right of form
5	package_type	enum('TDC','POC','Refresher')	utf8mb4	utf8mb4_unicode_ci	NO	NULL			
6	vehicle_category	enum('Automobile','Motorcycle','Tricycle')	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
7	transmission	enum('Auto','Manual')	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
8	status	enum('draft','submitted','approved','rejected')	utf8mb4	utf8mb4_unicode_ci	NO	draft			
9	personal_info	json	utf8mb4	utf8mb4_unicode_ci	NO	NULL			{emergency_contact:{}}
10	course_preferences	json	utf8mb4	utf8mb4_unicode_ci	NO	NULL			{schedule_pref:[], instructor_pref:null}
11	rejection_reason	text	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
12	reviewed_by	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	YES	NULL		users(id)	
13	reviewed_at	timestamp	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
14	created_at	timestamp	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
15	updated_at	timestamp	utf8mb4	utf8mb4_unicode_ci	YES	NULL			

Figure C.9 Enrollement_Forms

#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	NO	NULL	auto_increment		
2	enrollment_form_id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	YES	NULL		enrollment_forms(id)	
3	code	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL			
4	student_id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	NO	NULL		student_profiles(id)	
5	course_id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	NO	NULL		courses(id)	
6	instructor_id	bigint unsigned	utf8mb4	utf8mb4_unicode_ci	YES	NULL		instructor_profiles(id)	
7	status	enum('waiting_list','pending','active','completed','dropped')	utf8mb4	utf8mb4_unicode_ci	NO	pending			
8	progress_percent	decimal(5,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
9	final_grade	decimal(5,2)	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
10	final_result	enum('pass','fail')	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
11	remarks	text	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
12	start_date	date	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
13	target_completion_date	date	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
14	total_amount	decimal(10,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
15	amount_paid	decimal(10,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
16	balance	decimal(10,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
17	tdc_hours_completed	decimal(5,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
18	tdc_hours_required	decimal(5,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
19	pdc_hours_completed	decimal(5,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
20	pdc_hours_required	decimal(5,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
21	pdc_kms_driven	decimal(10,2)	utf8mb4	utf8mb4_unicode_ci	NO	0.00			
22	tdc_status	enum('not_started','in_progress','completed')	utf8mb4	utf8mb4_unicode_ci	NO	not_started			
23	pdc_status	enum('not_started','in_progress','completed')	utf8mb4	utf8mb4_unicode_ci	NO	not_started			
24	created_at	timestamp	utf8mb4	utf8mb4_unicode_ci	YES	NULL			
25	updated_at	timestamp	utf8mb4	utf8mb4_unicode_ci	YES	NULL			

Figure C.10 Enrollments



Name: instructor_performances Primary: id									
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	→ EMPTY
2	instructor_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	instructor_profiles(id)	→ EMPTY
3	student_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	student_profiles(id)	→ EMPTY
4	enrollment_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	enrollments(id)	→ EMPTY
5	booking_session_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	booking_sessions(id)	→ EMPTY
6	rating	int	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY
7	performance_criteria	json	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY
8	feedback_comment	text	utf8mb4	utf8mb4_unicode_ci	YES	NULL	EMPTY	EMPTY	→ EMPTY
9	areas_of_strength	text	utf8mb4	utf8mb4_unicode_ci	YES	NULL	EMPTY	EMPTY	→ EMPTY
10	areas_for_improvement	text	utf8mb4	utf8mb4_unicode_ci	YES	NULL	EMPTY	EMPTY	→ EMPTY
11	evaluation_date	date	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY
12	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY
13	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY

Figure C.11 Instructor_Performances

Name: lto_clinics Primary: id									
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	→ EMPTY
2	clinic_name	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY
3	accreditation_number	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY
4	address	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY
5	contact_number	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY
6	is_active	tinyint(1)	NULL	NULL	NO	1	EMPTY	EMPTY	→ EMPTY
7	accreditation_expiry	date	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY
8	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY
9	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY

Figure C.12 LTO_Clinics



Name	model_has_permissions	Primary	permission_id	model_id	model_type					
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	permission_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	permissions(id)	→	EMPTY
2	model_type	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→	EMPTY
3	model_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→	EMPTY

Figure C.13 Model_Has_Permissions

Name	model_has_roles	Primary	role_id	model_id	model_type					
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	role_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	roles(id)	→	EMPTY
2	model_type	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→	EMPTY
3	model_id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ EMPTY	EMPTY	→	EMPTY

Figure C.14 Model_Has_Roles



Name permissions		Primary id							
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	EMPTY
2	name	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	EMPTY
3	guard_name	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	EMPTY
4	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	EMPTY
5	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	EMPTY

Figure C.15 Permissions

Name role_has_permissions		Primary permission_id role_id							
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment
1	permission_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	permissions(id)	EMPTY
2	role_id	bigint unsigned	NULL	NULL	NO	NULL	EMPTY	roles(id)	EMPTY

Figure C.16 Role_Has_Permissions



Name roles		Primary id								
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	→ EMPTY	
2	name	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY	
3	guard_name	varchar(255)	utf8mb4	utf8mb4_unicode_ci	NO	NULL	EMPTY	EMPTY	→ EMPTY	
4	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	
5	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	

Figure C.17 Roles

Name system_metrics		Primary id								
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	NULL	NULL	NO	NULL	auto_increment	EMPTY	→ EMPTY	
2	metric_date	date	NULL	NULL	NO	NULL	EMPTY	EMPTY	→ EMPTY	
3	new_students	int	NULL	NULL	NO	0	EMPTY	EMPTY	→ EMPTY	
4	active_enrollments	int	NULL	NULL	NO	0	EMPTY	EMPTY	→ EMPTY	
5	completed_courses	int	NULL	NULL	NO	0	EMPTY	EMPTY	→ EMPTY	
6	total_bookings	int	NULL	NULL	NO	0	EMPTY	EMPTY	→ EMPTY	
7	revenue	decimal(15,2)	NULL	NULL	NO	0.00	EMPTY	EMPTY	→ EMPTY	
8	additional_data	json	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	
9	created_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	
10	updated_at	timestamp	NULL	NULL	YES	NULL	EMPTY	EMPTY	→ EMPTY	

Figure C.18 System_Metrics



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



Name		vehicles	Primary	id						
#	column_name	data_type	character_set	collation	is_nullable	column_default	extra	foreign_key	comment	
1	id	bigint unsigned	✓ NULL	NULL	NO	NULL	✓ auto_increment	EMPTY	→ EMPTY	
2	model	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
3	plate_number	varchar(255)	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
4	transmission	enum('auto','manual')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
5	type	enum('motorcycle','automobile','tricycle')	✓ utf8mb4	utf8mb4_unicode_ci	NO	NULL	✓ EMPTY	EMPTY	→ EMPTY	
6	status	enum('available','maintenance','in-use')	✓ utf8mb4	utf8mb4_unicode_ci	NO	available	✓ EMPTY	EMPTY	→ EMPTY	
7	maintenance_history	json	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
8	next_maintenance_date	date	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
9	created_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	
10	updated_at	timestamp	✓ NULL	NULL	YES	NULL	✓ EMPTY	EMPTY	→ EMPTY	

Figure C.19 Vehicles



D. Survey/Evaluation Forms Used During Testing

SYSTEM EVALUATION AND ACCEPTANCE FORM

Researchers: Team B1

Institution: Binalatongan Community College

I. PURPOSE OF THE EVALUATION

The purpose of this evaluation is to gather objective feedback from stakeholders and end-users regarding the functional and non-functional qualities of the developed system. Your participation will help validate the system's effectiveness in managing driving school enrollment, instructor matching, and administrative workflows.

Please mark the box that corresponds to your level of agreement (1-5) for each criterion.

Evaluation Criteria	Strongly Disagree (1)	Disagree (2)	Neutral (3)	Agree (4)	Strongly Agree(5)
The system is easy to navigate					
The interface design is clear and visually appealing					
System instructions and labels are understandable					
The system is user-friendly and requires minimal effort to learn					
The system provides clear feedback for every transaction.					

Table 1. Usability testing



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan
Email Address: bcc.sccp@yahoo.com



Evaluation Criteria	Strongly Disagree (1)	Disagree (2)	Neutral (3)	Agree (4)	Strongly Agree (5)
System Functionality works as expected					
System performance is fast and responsive					
The system meets my needs and requirements					
The interface design is clear and visually appealing.					
System instructions and labels are understandable.					

Table 2. User Acceptance Testing

RESPONDENT INFORMATION

Signature over printed name



E. Collected Sample Documents for Document Analysis/Data Gathering

This appendix presents the compiled collection of legacy forms, operational logbooks, and administrative records gathered by the proponents during the data gathering and system analysis phase of the study. These instruments served as the baseline for mapping out the manual workflow, transaction procedures, and informational bottlenecks of the current system. The structural layouts and data fields extracted from these documents directly guided the development of the database schema, input validations, and automated reporting modules of the proposed system.



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



ORIGINAL COPY

REGISTRATION FORM

REGISTRATION FORM BLRT Driving School

MSME Building

Rizal Street

San Carlos City, Pangasinan

Tel Nos.: 09614877999

09664152999

NAME _____
Surname First Name Middle Name

ADDRESS _____

OCCUPATION _____

EDUCATIONAL ATTAINMENT: ☐ Elem. ☐ High School ☐ College

BIRTHDATE _____

CIVIL STATUS _____ SEX _____

CONTACT NO. _____

EMAIL ADD: _____

LTMS Client ID NO. _____

STUDENT PERMIT/DRIVER LICENSE NO. _____

Control No.: _____

Date Enrolled: _____

PACKAGE:

() TDC (Theoretical Driving Course)

Schedule Date: _____

() PDC (Practical Driving Course)

() 4 Wheel Automatic Trans. (B-AT)

() 4 Wheel Manual Trans. (B-MT)

() Motorcycle Automatic (A-AT)

() Motorcycle Manual/Clutch (A-MT)

() Tricycle Manual/Clutch (A1-MT)

Release Date: _____

() Refresher Course

PAYMENT:

() Full Payment _____

() Partial _____

() Balance _____

RULES AND REGULATIONS FOR STUDENTS:

1. Tuition Fee shall be fully Paid on the first day of driving lesson.
2. Tuition Fees for TDC/PDC are Non-Refundable once paid.
3. Students who withdraw or wish to withdraw from the school, for any reason, are not entitled to a refund of any fees paid.
4. Enrollment Fees is non Transferable.
5. No food, smoking or cellphone during lesson proper.
6. Cancellation of Practical Driving shall be within 1 day before schedule, penalty of 150 pesos if no notice of cancellation.
7. Student under influence of liquor/drugs will not be given driving lesson.
8. Strictly NO WEARING OF SHORT PANTS, SANDOS, SLIPPERS.
9. Students on Practical Driving Lesson shall bring always S.P. or D.L. and must wear rubber shoes.
10. BLRT has the right to cancel lesson of non-observance of safety features and health protocols.
11. Practical Driving Course Certificate shall be available on specified release date.

THIS IS TO CERTIFY THAT I HAVE READ AND FULLY UNDERSTAND THE FOREGOING RULES STATED ABOVE

SIGNATURE OVER PRINTED NAME
STUDENT

SIGNATURE OVER PRINTED NAME
RECEIVING CLERK

NOTED BY: _____
School Administrator

Figure C.1: BLRT Driving School Manual Registration Form



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan
Email Address: bcc.sccp@yahoo.com



CALM LEARNING, CALM DRIVING



STUDENT NAME: _____

PRACTICAL DRIVING (ACTUAL) 8 HOURS.

Automobile Driving: () Automatic Transmission () Manual Transmission

Motorcycle: () With Clutch () Automatic () Tricycle

DATE _____ START TIME _____ END TIME _____ KMS. _____ NO. HOURS _____

DATE _____ START TIME _____ END TIME _____ KMS. _____ NO. HOURS _____

DATE _____ START TIME _____ END TIME _____ KMS. _____ NO. HOURS _____

TOTAL NO. OF HOURS _____

STUDENT SIGNATURE: _____ PRACTICAL DRIVING INSTRUCTOR _____

NOTED BY SCHOOL ADMINISTRATOR

NOTED BY SCHOOL ADMINISTRATOR

STUDENT SIGNATURE: _____ PRACTICAL DRIVING INSTRUCTOR _____

TOTAL NO. OF HOURS _____

DATE _____ START TIME _____ END TIME _____ KMS. _____ NO. HOURS _____

DATE _____ START TIME _____ END TIME _____ KMS. _____ NO. HOURS _____

DATE _____ START TIME _____ END TIME _____ KMS. _____ NO. HOURS _____

Automobile Driving: () Automatic Transmission () Manual Transmission
Motorcycle: () With Clutch () Automatic () Tricycle

PRACTICAL DRIVING (ACTUAL) 8 HOURS.

STUDENT NAME: _____



Figure C.2: BLRT Driving School Practical Driving Course (PDC) Training Log.



Binalatongan Community College

Brgy. Ilang, San Carlos City, Pangasinan

Email Address: bcc.sccp@yahoo.com



CALM LEARNING, CALM DRIVING



PRACTICAL DRIVING ASSESSMENT REPORT

Name of Student:	Signature:		
Vehicle Type:			
Date of Assessment:	Start Time:	End Time:	
Reason for Assessment:			

To be completed & carried out by LTO Accredited Driving Instructor ONLY

Important Note:

To pass, you must not have errors marked for Items under PRE-DRIVE CHECKLIST, no marks in the CRITICAL DRIVING ERROR section, and no more than 15 Errors/Poor marked for the Road Assessment Section.

2. PRE DRIVING CHECKLIST

2.1	Check/Switch, Lights and Windshield Wiper.	☐
2.2	Adjust Mirrors (Rear view mirror and Side mirror)	☐
2.3	Use Seatbelt/Helmet	☐
2.4	Check doors	☐
2.5	Disengage clutch when starting engine	☐
2.6	Disengage emergency/parking brake before moving	☐

1. IMMEDIATE FAILS (Any of the below)

CRITICAL DRIVING ERROR

1.1	Failure to stop at RED SIGNAL, STOP SIGN or STOP LINE.	☐
1.2	Colliding with any mobile or immobile object, or curb.	☐
1.3	Triggering Intervention of the instructor or causing any other motorist to swerve to avoid collision.	☐
1.4	Not Adhering to Traffic Sign (entering No-Entry Sign or Yellow Box Junction) or instructor's directions.	☐
1.5	Lack of Vehicle Control/Dangerous maneuver.	☐
1.6	Failure to ensure the road, lane or roundabout is clear before entering/proceeding.	☐
1.7	Speed/Lane violation.	☐

ON THE ROAD ASSESSMENT 1 = Poor / 2 = Fair / 3 = Good

3. DRIVING SKILLS

	Assessment Rating	1	2	3
3.1	Steering: Position of hands, hand grip, smoothness	☐	☐	☐
3.2	Engine Control: Smooth start up	☐	☐	☐
	Use of gears	☐	☐	☐
	Use clutch	☐	☐	☐
	Use of accelerator	☐	☐	☐
3.3	Use of Brakes: Apply smooth braking	☐	☐	☐
	Reactions to Hazards	☐	☐	☐
	Vehicle/Motor turning	☐	☐	☐
3.4	Speed Control: Observance of speed limit	☐	☐	☐
	Observance of traffic rules	☐	☐	☐
	Road signs knowledge	☐	☐	☐
3.5	Maneuvering: Turning left/right/U-turn	☐	☐	☐
	Takes proper lane and Signal intention	☐	☐	☐
	Use of Hand and Light Signal	☐	☐	☐
	Backing/Parking: Control of the Vehicle	☐	☐	☐
	Correct Spacing/Distance	☐	☐	☐
	Number of Attempts	☐	☐	☐

4. OBSERVANCE TO TRAFFIC RULES

	Assessment Rating	1	2	3
3.1	Right of Way: For other Vehicles	☐	☐	☐
	For Pedestrian, Emergency and Bicyclist	☐	☐	☐
3.2	Road signs knowledge	☐	☐	☐
	In Changing Lanes & Yielding	☐	☐	☐
	While approaching Intersection	☐	☐	☐
3.3	In Passing/Being Passed	☐	☐	☐
	Stop Lights/Signals & Others:	☐	☐	☐
	Obey Traffic Signs	☐	☐	☐
3.4	Position After Stopping	☐	☐	☐
	Making Full Stop when Necessary	☐	☐	☐
	Anticipating before Signal Changes	☐	☐	☐
3.5	Exercise due care for Pedestrian	☐	☐	☐

4. OVERALL ASSESSMENT

1	Slow Learner
2	Fast Learner
3	Accompanied by Hesitancy
4	Recommended for Additional Actual Driving

5. REMARKS AND OBSERVATIONS:

Name of Instructor:	Noted by School Administrator:
Signature:	Signature:
Date:	Date:

Figure C.3. BLRT Driving School Practical Driving Assessment Report.



CABUANG, JOSHUA Q.

joshuacabuang0@gmail.com | 09361498100 | Abanon San Carlos City,
Pangasinan | <https://github.com/dumbDev6969>



PERSONAL PROFILE

An incoming entry-level Full-Stack Developer and graduating BSIT student with deep foundational competencies in full-stack engineering, secure database architecture, and automated system workflows. Possesses a rigorous technical background developed through leading complex, large-scale capstone projects from conceptualization to deployment. Highly adept at translating manual business operations into optimized digital solutions, managing data integrity, and utilizing modern development tools. A proactive problem-solver prepared to contribute immediate technical value to development teams.

EDUCATIONAL BACKGROUND

Primary	:	Abanon Elementary School
Secondary	:	Abanon National Highschool
Tertiary	:	Binalatongan Community College

PROJECT(S)

Title: Web-based Intelligent Driving School Management and Enrollment System

Role: Lead Full-Stack Developer | *February 2026 – May 14, 2026 Present* End-to-End Development: Engineered the entire system architecture using the TALL stack (Tailwind CSS, Alpine.js, Laravel, and Livewire), transitioning a legacy driving school workflow into a centralized digital application.

TECHNICAL SKILLS & COMPETENCIES

Full-Stack Frameworks (TALL Stack): Laravel, Livewire, Alpine.js, Tailwind CSS

Core Programming & Languages: PHP, JavaScript, HTML5, CSS3

Database Systems: MySQL (Schema Design, Optimization, TablePlus)

Tools & Version Control: Git, GitHub, VS Code, Laravel Herd

Productivity & Documentation: Google Suite (Docs, Sheets, Slides), Technical Documentation



JACLA, RYAN M.

jaclaryan17@gmail.com | 09770587826 | Pangalangan San Carlos City, Pangasinan



PERSONAL PROFILE

An incoming entry-level UI/UX Designer and graduating IT student with deep foundational competencies in digital prototyping, user interface design, and visual communication. Possesses a rigorous technical background developed through designing interactive system layouts and user journeys from concept to high-fidelity wireframes. Highly adept at translating business requirements into intuitive, user-friendly designs using Figma and Canva, while efficiently managing project data and collaborative workflows using the Microsoft productivity suite (Excel, PowerPoint, and Teams). A proactive problem-solver prepared to contribute immediate creative and technical value to development teams.

EDUCATIONAL BACKGROUND

Primary	:	Tarcan Elementary School
Secondary	:	Aisat Bulacan National Highschool
Tertiary	:	Binalatongan Community College

PROJECT(S)

Title: Web-Based Intelligent Driving School Management and Enrollment System **Role:** Lead UI/UX Designer | February 2026 – May 14 2026 **UI/UX Prototyping:** Architected the complete user interface, user journeys, and responsive wireframes in Figma, transforming complex legacy driving school workflows into clean, intuitive, high-fidelity digital interfaces. Designed interactive system mockups and branding elements using Canva to guarantee visual consistency. Collaborated closely with the development team to ensure pixel-perfect asset handoff and a seamless user experience across the enrollment, scheduling, and analytics modules.

TECHNICAL SKILLS & COMPETENCIES

UI/UX & Prototyping Tools: Figma, Canva

Core Design Competencies: User Interface (UI) Design, High-Fidelity Wireframing, User Journeys, Interactive Prototyping, Visual Branding

Productivity & Documentation: Google Suite (Docs, Sheets, Slides), Technical Documentation



CASAY, JIM SPENCER LEE C. CASAY

jimspencerleec@gmail.com | 09458145165 | Calobaoan San Carlos City, Pangasinan



PERSONAL PROFILE

An incoming entry-level UI/UX Designer and graduating IT student with deep foundational competencies in digital prototyping, user interface design, and visual communication. Possesses a rigorous technical background developed through designing interactive system layouts and user journeys from concept to high-fidelity wireframes. Highly adept at translating business requirements into intuitive, user-friendly designs using Figma and Canva, while efficiently managing project data and collaborative workflows using the Microsoft productivity suite (Excel, PowerPoint, and Teams). A proactive problem-solver prepared to contribute immediate creative and technical value to development teams.

EDUCATIONAL BACKGROUND

Primary	:	Calobaoan Elementary School
Secondary	:	Abanon National High School
Tertiary	:	Binalatongan Community College

Title: Web-Based Intelligent Driving School Management and Enrollment System **Role:** Lead UI/UX Designer | February 2026 – May 14 2026 **UI/UX Prototyping:** Architected the complete user interface, user journeys, and responsive wireframes in Figma, transforming complex legacy driving school workflows into clean, intuitive, high-fidelity digital interfaces. Designed interactive system mockups and branding elements using Canva to guarantee visual consistency. Collaborated closely with the development team to ensure pixel-perfect asset handoff and a seamless user experience across the enrollment, scheduling, and analytics modules.

TECHNICAL SKILLS & COMPETENCIES

UI/UX & Prototyping Tools: Figma, Canva, Drow.io

Design Competencies: User Interface (UI) Design, High-Fidelity Wireframing, User Journeys, Interactive Prototyping, Visual Branding

Productivity & Documentation: Google Suite (Docs, Sheets, Slides), Technical Documentation



PALISOC, ROMARK D.

romarkpalisoc59@gmail.com | 09125613423 | Poblacion Urbiztondo Pangasinan



PERSONAL PROFILE

An incoming entry-level Full-Stack Developer and graduating BSIT student with deep foundational competencies in full-stack engineering, secure database architecture, and automated system workflows. Possesses a rigorous technical background developed through leading complex, large-scale capstone projects from conceptualization to deployment. Highly adept at translating manual business operations into optimized digital solutions, managing data integrity, and utilizing modern development tools. A proactive problem-solver prepared to contribute immediate technical value to development teams.

EDUCATIONAL BACKGROUND

Primary	:	Urbiztondo Integrated School
Secondary	:	Urbiztondo Catholic School
Tertiary	:	Binalatongan Community College

PROJECT(S)

Title: Web-based Intelligent Driving School Management and Enrollment System

Role: Lead Full-Stack Developer | *February 2026 – May 14, 2026 Present* End-to-End Development: Engineered the entire system architecture using the TALL stack (Tailwind CSS, Alpine.js, Laravel, and Livewire), transitioning a legacy driving school workflow into a centralized digital application.

TECHNICAL SKILLS & COMPETENCIES

Full-Stack Frameworks (TALL Stack): Laravel, Livewire, Alpine.js, Tailwind CSS

Core Programming & Languages: PHP, JavaScript, HTML5, CSS3

Database Systems: MySQL (Schema Design, Optimization, TablePlus)

Tools & Version Control: Git, GitHub, VS Code, Laravel Herd

Productivity & Documentation: Google Suite (Docs, Sheets, Slides), Technical Documentation